

A Comparative Analysis of Memory Management, Concurrency, and Performance in Edge-AI

Timothy Clarke

University of Dundee

United Kingdom

2712139@dundee.ac.uk

Abstract

This dissertation presents a systems-engineering comparison of C++20, Rust 1.97.1, and CPython 3.10.12. A functionally identical tri-stream Human Activity Recognition pipeline was implemented in each language on an NVIDIA Jetson Orin Nano, utilising a zero-allocation architecture. The implementations were evaluated under varying ingestion rates, backpressure policies, and hardware power constraints to isolate runtime latency, maximum throughput, and thermal degradation.

The evaluation revealed that C++ and Rust achieve similar performance results using the Jetson's unconstrained power mode, sustaining ingestion rates up to 5.5 times the native sensor rate with no dynamic memory allocation when using Exponential Backoff. However, both compiled languages were unable to sustain ingestion rates above 4.0 when using the Bounded Queue policy due to lock contention. In unconstrained mode, Python reached terminal saturation at 4% of the native rate, though this slightly improved to 6% under the 7-Watt profile. Statistical analysis ruled out garbage collection as the cause of Python's poor performance, indicating Global Interpreter Lock contention as the primary bottleneck.

Analysis of the backpressure policies revealed a trade-off: flow-control policies prevent data loss but suffer from latency deadline breaches at terminal saturation, whereas load-shedding policies accept data loss to guarantee deadline adherence. Alternatively, Adaptive Decimation attempts to prevent saturation and retain temporal continuity, but at the expense of higher data loss even at moderate ingestion rates that the pipeline could otherwise sustain without data loss or deadline breaches. Furthermore, the evaluation showed that deploying the compiled implementations under a constrained 7-Watt power profile increases latency degradation when using flow-control policies due to limited computational resources, confirming that the performance of Edge-AI pipelines is impacted by the power constraints of the hardware.

This study recommends Rust for real-time Edge-AI deployments. It matches the execution speed of C++ while providing compiler-enforced memory safety, significantly reducing the maintenance overhead of complex concurrent systems.

1. Introduction

1.1. Background and Context

In recent years, Edge Artificial Intelligence (Edge-AI) has begun to move the deployment of AI models from centralised cloud-based servers to local devices, such as sensors, mobile phones, and other embedded systems. This decentralisation of AI processing offers several advantages over traditional cloud computing: (1) it enables *real-time processing* by removing network latency, which is essential for time-sensitive applications such as autonomous vehicles and industrial automation; (2) it drastically *reduces internet bandwidth usage*, allowing systems to operate in environments where connectivity is unreliable or unavailable, such as remote weather stations or satellite image analysis; (3) it enhances *privacy and security* by ensuring that sensitive information, such as healthcare monitoring or smart home video feeds, is processed locally and not transmitted across the internet; and (4) by removing the need for continuous data transmission, Edge-AI can improve overall *energy efficiency* in battery-powered Internet of Things (IoT) deployments. These advantages are driving the expansion of large-scale Edge-AI projects, such as in smart city infrastructure, where Edge-AI is increasingly deployed directly into municipal systems to improve efficiency and sustainability in areas like the control of "smart" traffic lights or street lighting.

While recent advancements in heterogeneous System-on-Chips (SoCs) have made Edge-AI deployments practical by integrating dedicated AI accelerators into small form factors, these devices do bring challenges in terms of resource constraints, such as limitations in memory, power availability, and heat dissipation. Furthermore, remote software updates to maintain Edge-AI applications also come at a cost, as they can be expensive and time-consuming, especially when dealing with a large number of devices. The software must make maximum use of the limited hardware resources, while also remaining stable over the long term, making programming language selection an important design decision for Edge-AI pipelines.

1.2. Problem Statement

A language's runtime model dictates memory management and thread synchronisation under load, which directly impacts latency, throughput, and resource usage. For example, manual memory management offers fine-grained control and increased performance, but simultaneously increases the risk of memory leaks and undefined behaviour. Conversely, memory management may be automated through Garbage Collection (GC) at the cost of execution overhead and unpredictable latency jitter.

This dissertation focuses on the performance metrics at the system level. *Latency* does not refer to network transmission time, but rather the processing time from when the sensor data is created to when the final AI prediction is completed. This includes queueing delays, inference time, and final fusion of the prediction. A deadline of 100 ms is chosen for the tri-stream HAR pipeline, based on what Xue et al. [1] identified as the maximum allowable latency for effective real-time coaching feedback. *Throughput* measures how many sensor events the pipeline can process per unit of time (e.g. per second) when under sustained load.

Selecting a programming language for Edge-AI pipelines is often guided by developer familiarity or generalised benchmarks, rather than the evaluation of runtime models under stress with the constraints of embedded hardware. When deploying on Edge hardware, the aforementioned challenges amplify the impact of programming language choice. There is a lack of empirical evidence of how specific memory management and concurrency models interact with backpressure policies under heavy, fluctuating loads. Additionally, resource contention and other confounders can introduce noise that limits the validity of naive comparisons.

This dissertation addresses this gap by providing empirical evidence and an evaluation of pipelines deployed on resource-constrained hardware, with a focus on the trade-offs among C++20 (with GCC 15.2.0), Rust (1.97.1), and Python (CPython 3.10.12) implementations of a tri-stream Human Activity Recognition (HAR) pipeline on industry-standard Edge-AI hardware. It focuses on three variables:

(1) language runtime models, (2) backpressure policies under various ingestion rates, and (3) hardware power constraints.

1.3. Research Questions and Objectives

The primary research questions are:

- RQ1 Runtime Performance:** How do the runtime models of C++, Rust, and Python influence latency, throughput, and memory consumption in a tri-stream HAR pipeline on Edge-AI hardware?
- RQ2 Backpressure Interaction:** How do the runtime models interact with different backpressure policies under varying load, and what are the trade-offs in observed deadline adherence, system stability, and allocator/GC pressure and concurrency overhead?
- RQ3 Dynamic Profiling vs. Runtime Behaviour:** To what extent do dynamic memory/concurrency profiling metrics explain the observed performance bottlenecks and trade-offs under load?

The primary research objectives are:

- RO1** Implement a functionally identical tri-stream HAR pipeline in C++, Rust, and Python, ensuring optimised idiomatic implementations for each language.
- RO2** Evaluate the performance of each implementation under controlled conditions, using a shared deterministic load generator, to quantify how backpressure policies and runtime models impact latency, throughput, and memory consumption under both constrained and unconstrained hardware power profiles.
- RO3** Analyse runtime model overhead to explain performance differences, and derive empirically grounded guidance for language selection in constrained Edge-AI deployments.

1.4. Research Contributions

This dissertation offers the following contributions to software engineering for Edge-AI systems:

- C1 Cross-Language Runtime Evaluation:** Addressing RQ1, this work delivers an empirical comparison of how C++, Rust, and Python runtime models (memory management and concurrency) influence latency, throughput, and memory consumption on resource-constrained embedded hardware.
- C2 Interaction Analysis:** Addressing RQ2, this work provides a controlled assessment of how backpressure policies interact with runtime models under various loads, and identifies trade-offs between throughput and long-term system stability.
- C3 Root-Cause Identification:** Addressing RQ3, this work presents empirical evidence linking dynamic memory allocation churn, GC pause duration, and concurrency overhead to system latency and throughput.
- C4 Evidence-Based Guidelines:** Addressing findings across all research questions, this dissertation offers empirically grounded recommendations for language selection in real-time, multi-stream edge deployments.

1.5. Scope and Limitations

The focus of this dissertation is on the interaction of three language runtime models (C++, Rust, and Python) with system latency and throughput, using idiomatic implementations of a tri-stream HAR pipeline on industry-standard Edge-AI hardware. The scope is limited to an implementation representative of real-world multi-modal processing, with the experimental variables limited to the choice of language runtime model, the applied backpressure policy, and the hardware power profile.

The pipeline architecture, deterministic load generator, and backpressure implementations are designed for portability across Linux environments. However, AI acceleration, thermal behaviour, and power management are SoC-dependent. Consequently, the profiling and acceleration tools used during evaluation (e.g. NVIDIA tegras-tats, TensorRT) are specific to the Jetson Orin Nano platform and cannot be directly applied across other edge devices.

A further limitation of this study is that it is not concerned with the accuracy of the HAR prediction results. To ensure the data is both deterministic and reproducible for each implementation, a synthetic load generator is used. Pre-recorded data is not used to avoid confounding the results with disk I/O bottlenecks, ensuring the data is available to the pipeline at a consistent rate. It is acknowledged that the choice of backpressure policy would impact prediction accuracy and usefulness of the results, due to the dropping of data and temporal discontinuity. However, this dissertation is concerned with performance measurements of latency, throughput, and memory efficiency at the system level, and not the accuracy of the HAR model itself.

1.6. Dissertation Outline

The remainder of this dissertation is structured as follows: **Chapter 2** provides background on the evolution of Edge-AI and the heterogeneous devices employed. It also introduces the programming languages evaluated in this dissertation, and briefly explains the concepts of AI acceleration, backpressure, and load shedding. **Chapter 3** reviews related literature regarding language efficiency benchmarks, the trade-offs between memory safety and cognitive load, backpressure policies, and measuring latency, concluding by identifying the research gap. **Chapter 4** details the methodology, including the hardware and software setup, backpressure interfaces, and profiling toolchains used to collect metrics. **Chapter 5** describes the implementation of the HAR pipeline in each language, highlighting language-specific optimisations and challenges. **Chapter 6** presents the results, including performance metrics, memory allocation and GC pressure, and backpressure outcomes under varying loads. **Chapter 7** discusses the findings and limitations of the study. Finally, **Chapter 8** concludes by addressing the research questions, providing practical recommendations for language selection in Edge-AI contexts, and suggesting directions for future work.

2. Background

2.1. The Evolution of Edge Computing

The late 1990s saw a growth of online multimedia that demanded a solution to tackle increased network congestion and latency. Karger et al. [2] proposed a concept of *distributed caching protocols* that evolved into modern-day *Content Delivery Networks* (CDNs), ensuring that static content could be cached on servers located closer to end-users, thus reducing latency and bandwidth usage, especially during periods of high demand.

The release of the iPhone in 2007, followed by the Android operating system in 2008, and the Windows Phone in 2010, marked a rapid increase in the use of mobile devices, and created user demand for computationally intensive applications. However, as Satyanarayanan et al. [3] established, “considerations such as weight, size, battery life, ergonomics, and heat dissipation exact a severe penalty in computational resources such as processor speed, memory size, and disk capacity.” To bypass these physical limitations, they took the concept of CDNs further by proposing decentralised and widely dispersed *cloudlets* — servers located on the network edge and close to end-clients (e.g. in cafe premises). By running customised service software using hardware VM technology, cloudlets allow mobile devices to act as thin clients. This overcomes hardware constraints without unacceptable latency and bandwidth usage that would be introduced if remote cloud servers were used.

The next decade saw an explosive growth of the IoT, fuelled by the adoption in areas such as fitness wearables (the first Fitbit Tracker launched in 2009), smart home Devices (Google acquired Nest Labs in 2014, and Amazon acquired Ring LLC in 2018), and “dockless” bicycle-sharing schemes (Lime launched in 2017). Remote devices were no longer just data consumers, but had also become data producers. Shi et al. [4] defined “edge” not as a specific device, but as any computing and networking resource along the path between the data source and the data centre. They recognised that the data bandwidth and centralised processing in traditional cloud computing were bottlenecks, arguing that data should be processed at the proximity of the data source.

At the same time, the integration of AI rapidly accelerated. While some applications were designed to run on remote cloud servers (e.g. ChatGPT, launched in late 2022), latency-critical applications depended on local-device processing to ensure safety and reduce dependence on available network bandwidth (e.g. Tesla Autopilot, launched in 2014, and the Waymo One in 2018).

This transition to *Edge-AI* required overcoming the hardware obstacles identified by Satyanarayanan et al. [3] and the network bottlenecks identified by Shi et al. [4]. Building on this, Zhou et al. [5] provided a comprehensive survey of recent research efforts in Edge Intelligence, and identified that physical proximity to the data source is critical to reducing monetary costs, latency, and the risk of privacy leakage. For evaluating the quality of Edge-AI inference, they highlighted latency, accuracy, energy consumption, privacy, and memory footprint. While communication overhead is eliminated by offline edge processing, latency, energy consumption, and memory overhead remain relevant to this dissertation. For example, backpressure policies may intentionally drop data to ensure system stability, sacrificing model accuracy to satisfy latency deadlines.

2.2. Heterogeneous Devices and Power Constraints

Heterogeneous devices combine different types of processing units, such as a CPU and a GPU, onto a single chip and are an increasingly common solution for Edge-AI deployments. Modern heterogeneous Systems-on-Chips (SoCs), such as the NVIDIA Jetson Orin Nano, integrate dedicated Compute Unified Device Architecture (CUDA) cores for general-purpose GPU computing, and Tensor cores for AI acceleration. These allow optimised engines like TensorRT to efficiently execute AI pipelines, such as HAR, locally without depending on remote cloud servers. Furthermore, they support the enabling technologies identified by Zhou et al. [5], such as model compression (e.g. quantisation and pruning), to maximise inference speed.

However, embedded devices with a small form factor generate significant heat when under sustained heavy loads, such as that generated by Edge-AI pipelines. To prevent hardware failure, the Jetson Orin Nano relies on Dynamic Voltage and Frequency Scaling (DVFS) [6] to reactively throttle the CPU and GPU speeds when thermal limits are reached. Furthermore, to adhere to power budgets, devices employ static power profiles that cap maximum clock frequencies. Peluso et al. [7] demonstrated that dynamic throttling introduces non-deterministic pipeline performance degradation, while power capping starves computational throughput. It can be concluded that to minimise premature throttling, the software architecture and language runtime model must be efficient by minimising unnecessary CPU and memory overhead.

2.3. AI Acceleration

To deploy AI models on resource-constrained edge devices, they must first be compiled into a format that is optimised for the target AI acceleration hardware. A software stack on the device is responsible for executing the model, while the client, or *host*, manages the data transfer and inference orchestration.

As visualised in Figure 1, **CUDA** [8] provides a parallel execution environment and programming model for heterogeneous comput-

ing systems with NVIDIA GPUs, using a Single Instruction Multiple Threads (SIMT) architecture. In CUDA, the CPU is referred to as the *host*, and the GPU is referred to as the *device*. CUDA clients are responsible for managing the transfer of data between *host memory* and *device memory*.

CUDA allows developers to write a *kernel* function that is launched asynchronously from the host code and executed in parallel across many threads (using different data) on the device, enabling high-performance computing for AI workloads. The kernel *threads* are grouped into *thread blocks*, which in turn are grouped into a *grid*. Each thread block is split into *warps* of 32 threads that are executed in lockstep on a single device *Streaming Multiprocessor*.

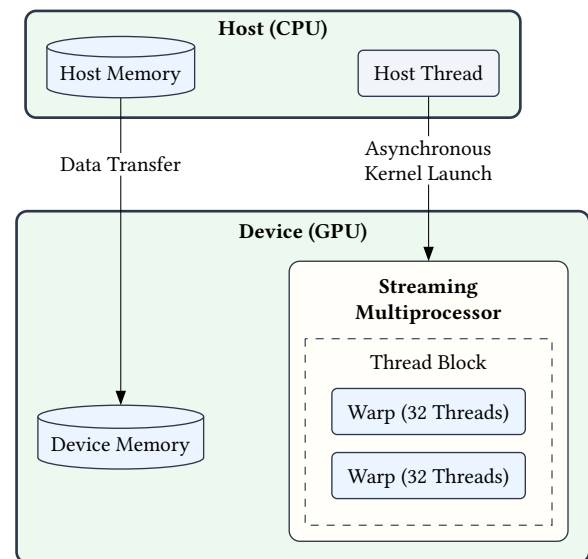


Figure 1: CUDA architecture overview showing the host (CPU) and device (GPU) memory separation, and the asynchronous launch of kernel functions to the device.

CUDA Deep Neural Network (**cuDNN**) [9] is a GPU-accelerated library of primitives for deep neural networks, that sits on top of CUDA and runs on the device to provide higher-level abstractions and optimised implementations of common deep learning operations (e.g. normalisation, matrix multiplication, softmax, etc.).

TensorRT [10] is responsible for compiling an Open Neural Network Exchange (**ONNX**) [11] model into an optimised, hardware-specific execution engine. When the model is loaded by ONNX Runtime, this compiled engine can be cached to disk as an Execution Provider Context (`_epctx.onnx`) file, preventing the need for recompilation and decreasing startup times on the Jetson GPU.

2.4. Language Runtimes & Memory Models

To maximise computational throughput under the power and resource constraints inherent in Edge-AI hardware, the pipeline implementation must be highly efficient. This dissertation compares the impact of three language runtime models on Edge-AI performance: C++, Rust, and Python.

C++ is an Ahead-of-Time (AOT) compiled (i.e. compiled to native machine code before execution) general-purpose language, designed for high performance and efficiency. It provides manual memory management, allowing fine-grained control over usage. However, this introduces risks of severe memory-safety bugs such as double-free and use-after-free, which can lead to undefined behaviour and security vulnerabilities.

Rust is similarly an AOT-compiled general-purpose language. In addition to high performance and efficiency, it also provides memory safety. Rust achieves this by employing Ownership-Based Resource Management (OBRM, more commonly referred to as its ownership and borrowing model) which provides compile-time guarantees of

memory safety — every value has a single owner, and the compiler ensures that value references do not outlive their owners. This provides a strong guarantee of memory correctness, but potentially increases cognitive load as the OBRM is a novel concept that introduces a steep learning curve.

Python is a general-purpose language that is compiled to bytecode and interpreted at runtime. It emphasises simplicity and ease of both writing and reading. CPython (the reference implementation) has a Foreign Function Interface (FFI) that allows it to interface with other languages, such as C/C++ libraries. Python is often used to automate tasks and for data analysis and machine learning. It utilises a GC, abstracting memory management to reduce cognitive load and the risk of memory-safety bugs, but at the cost of increased latency and unpredictable jitter due to “stop-the-world” GC events which pause the executable’s threads to safely clean up memory. Latency is further impacted by CPython’s Global Interpreter Lock (GIL), which prevents true concurrency across multiple CPU cores by ensuring only one thread executes Python bytecode at any given time. However, C-extensions (such as ONNX) are not constrained by the GIL, as they can release the lock when performing long-running operations, allowing underlying threads to execute concurrently on multiple cores.

2.5. Stream Processing & Backpressure

In stream processing systems, such as HAR pipelines, data is generated continuously and must be processed in real-time. When the rate of generation exceeds the system’s processing capacity, backpressure builds up within the pipeline as the number of unprocessed data items increases. This can lead to memory exhaustion and system instability if not managed effectively.

However, because the pipeline cannot request physical sensors to slow down their rate of transmission, software-level backpressure policies are necessary. When internal buffers reach capacity, these policies typically employ one of two strategies: flow-control mechanisms that temporarily pause ingestion to preserve data, or load-shedding techniques that discard data (e.g. dropping the oldest, newest, or every n -th frame). While load shedding guarantees system stability and deadline adherence, it introduces a trade-off as temporal continuity is lost.

3. Literature Review

3.1. Benchmarking Language Efficiency

Empirical evaluations of programming languages highlight a trade-off between execution speed, energy consumption, and memory footprint. In a comprehensive study of 27 programming languages, Pereira et al. [12] showed that compiled languages typically were the most performant, needed less memory, and were more energy efficient. Conversely, interpreted languages required the most memory, consumed the most energy, and were the slowest.

However, the reported results also indicated that execution speed and energy efficiency do not perfectly correlate with memory efficiency. For example, in the normalised results, Rust performed second only to C in terms of energy efficiency (1.03) and execution speed (1.04), but seventh (1.54) in terms of memory usage.

3.2. Memory Safety vs. Cognitive Load

While languages like C++ offer high performance, their reliance on manual memory management introduces severe security vulnerabilities. The scale of this risk is reflected in the 2025 Common Weakness Enumeration (CWE) Top 25 Most Dangerous Software Weaknesses [13], where memory-safety flaws such as out-of-bounds writes accounted for seven (28%) of the top 25 exploits.

To address this, Rust’s OBRM provides compile-time guarantees of memory safety without a GC. Xu et al. [14] analysed 186 real-world bug reports in Rust projects to determine how effectively

OBRM prevents memory-safety bugs in practice. They found that all memory-safety bugs in the dataset, except one that was a compiler bug, were caused by developers using the `unsafe` keyword to bypass the compiler’s memory safety checks. However, while Coblenz et al. [15] found that developers generally understood the *concept* of ownership, they struggled with the *semantics* of references and borrowing, introducing a trade-off between memory safety and developer cognitive load.

3.3. Backpressure Policies

In Edge-AI pipelines, data generated by sensors may be produced at a faster rate than the hardware can process. Because real-time physical sensors emit data continuously and cannot be paused, pipelines must implement software-level backpressure. While flow-control mechanisms can buffer temporary spikes in data generation, sustained overload leads to memory exhaustion or latency degradation. Consequently, to maintain system stability and adhere to real-time deadlines, load-shedding techniques are often required.

Recent novel research has focused on techniques that dynamically drop frames based on the load’s content to maximise efficiency without degrading model accuracy. For example, Li et al. [16] proposed filtering techniques that discard frames based on video-frame deltas, shedding data when visual scene changes are minimal. Similarly, Rivetti et al. [17] explored load shedding based on the estimated execution duration of the load using a cost model built and maintained at run-time, intentionally bypassing processing that would cause the latency threshold to be violated.

3.4. Coordinated Omission

If a data-processing pipeline evaluation only starts to measure latency when processing an event begins instead of when an event truly occurs, or when a producer is stalled due to the lack of the consumer’s readiness to process data, it risks a phenomenon identified by Tene [18] as *Coordinated Omission*. This failure to record the true time that an event occurs means that queue delays (which may occur because of OS context switches, garbage collection pauses, or thermal throttling) are not recorded, consequently reducing latency measurements. Furthermore, because fewer events are recorded when the system is throttled or stalled, low-latency events form the majority of the recorded dataset, making a system appear more performant than it actually is.

Tene also warns against ignoring events beyond the 99th percentile, as doing so fails to expose the frequency and impact of systematic delays. While Tene advises that extreme percentiles (e.g. 99.9th and 99.99th) should be measured, this dissertation captures only to the 99.9th percentile due to the limited number of events ($< 10,000$) that are generated during a one-second epoch.

3.5. The Research Gap

While studies such as Pereira et al. [12] have evaluated language efficiency, these benchmarks are typically conducted on standard desktop-class hardware. In contrast, studies that have evaluated the performance of Edge-AI pipelines [5], [7] have focused on the AI models themselves, ignoring how the host programming language impacts performance through its memory churn, GC pauses, and concurrency overhead.

At the time of the study conducted by Pereira et al., Rust’s default memory allocator on some platforms (including the system used by the authors) was `jemalloc` [19], which is designed for fast concurrent execution on multi-processor systems by maintaining multiple memory arenas. However, the Core Rust team acknowledged several drawbacks of using `jemalloc`, including adding ~300KB to binary sizes. Following RFC 1974 [20], which allowed developers to override the global allocator, Rust 1.32.0 [21] changed from `jemalloc` to the standard system allocator. This necessitates a re-evaluation

of Rust's memory footprint, as the change in allocator may have impacted its memory and performance efficiency.

Studies that have proposed novel load-shedding techniques [16], [17] have not evaluated the impact of the runtime model on the performance of the backpressure policies, instead treating the programming language as zero-cost. For example, there is no evaluation of how the memory model of the language impacts the performance of policies when data is dropped, such as whether Python's GC introduces jitter when reclaiming memory.

Therefore, there is a research gap in empirically evaluating how the C++, Rust, and Python runtime models interact with backpressure policies under the hardware resource and power constraints of Edge-AI devices with continuous streams of sensor data. Consequently, capturing true latency measurements at high percentiles under heavy load is necessary to determine their suitability for Edge-AI pipelines.

4. Methodology

4.1. Hardware Stack

The NVIDIA Jetson Orin Nano (8GB Edition) was utilised as the target Edge-AI platform. It is a high-performance heterogeneous computing platform designed for Edge-AI development in embedded systems, allowing complex AI workloads and multi-stream pipelines to be run efficiently.

The developer kit includes a carrier board with I/O interfaces (e.g. USB, Ethernet, DisplayPort), a MicroSD card slot for initial booting without an NVMe SSD, and the Jetson Orin Nano module itself. This provides the following specifications [22], [23]:

- 6-core Arm Cortex-A78AE 64-bit CPU (clocked to 1.7 GHz) for general-purpose concurrent processing
- up to 67 Tera Operations Per Second (TOPS) of AI performance
- 8 GB of 128-bit LPDDR5 memory with a bandwidth of 102 GB/s
- 1024 CUDA cores for general-purpose GPU computing
- 32 Tensor cores for AI acceleration
- L1 and L2 caches with 64-byte cache lines

The pipeline architecture was designed to process data from a Waveshare IMX219-160 Camera Module [24], configured to capture 1920x1080 RGB frames at 30 FPS via Mobile Industry Processor Interface Camera Serial Interface 2 (MIPI CSI-2). Inertial measurement data was modelled on a Bosch Sensortec BMI088 Inertial Measurement Unit (IMU) Shuttle Board 3.0 [25], which combines a 3-axis accelerometer (1.6 kHz) and a 3-axis gyroscope (2.0 kHz) connected via Serial Peripheral Interface (SPI). While these physical sensors dictated the architectural data structures, ingestion frequencies, and shared memory boundaries, the physical modules themselves were substituted for the deterministic load generator during evaluation due to hardware attrition.

To ensure that disk I/O did not cause bottlenecks or confound performance comparisons, all implementations were executed from a 1TB Samsung 990 PRO PCIe 4.0 NVMe M.2 SSD [26]. A SanDisk "High-Endurance" microSD Card (64GB, Class 10/U3) [27] was only used for initial device installation, and was physically removed after the NVMe SSD was installed.

4.2. Software Stack

The Jetson was already flashed with NVIDIA's JetPack 6.2 SDK [28], which unlocks the "Super" performance tier by providing the uncapped MAXN_SUPER power profile, that enables the highest number of CPU/GPU cores and maximum clock frequencies across the SoC. The software stack includes Jetson Linux 36.4.3 (featuring the Linux Kernel 5.15 and an Ubuntu 22.04-based root file system) and CUDA 12.6.10, cuDNN 9.3.0, and TensorRT 10.3.0 for AI inference and acceleration. The software stack versions were selected as the most recent stable releases at the time of development, and were used for all implementations to ensure a consistent baseline for comparison.

The Jetson Linux operating system utilises the GNU C Library (glibc) memory allocator, which in turn is used by the installed C++ and Rust toolchains as they rely on the system allocator. This enables use of glibc-specific tools for analysis of memory fragmentation.

All implementations interact with the ONNX Runtime 1.24.0 to load and execute the AI model, which is responsible for the data transfer and inference orchestration on the host, and hands off responsibility to TensorRT for inference execution on the device.

Performance and overhead of the language runtime models was measured at the boundary of the host/device memory separation, where the CPU manages the runtime model, and the SIMT architecture runs the AI workloads on the GPU. This prevents confounding the results with hardware latency, and provides a clearer comparison of how each language's runtime model performs under load and backpressure.

A Docker container, based on NVIDIA's official *l4t-jetpack* image (36.4.0 [29]), was used to prevent host updates and ensure that the software toolchains and environment variables remained consistent for all implementations. While Docker introduces some performance overhead, it was considered acceptable to ensure a consistent and reproducible environment for all implementations. Five Docker arguments were used for every container: `--ipc=host` to allow access to the host's shared memory, `--privileged` and `--runtime=nvidia` to allow access to the GPU and Jetson device nodes, `--cap-add=SYS_NICE` to prevent timing jitter by allowing real-time scheduling, and `--volume=$(pwd)/results:/results` to allow the container to write results to the host file-system.

The generator was detached, using the Docker argument `--detach`, so that it would execute in the background. To prevent context-switching overhead and resource contention, the generator was pinned to the highest available core using its `--core` argument (core 5 in the unconstrained MAXN_SUPER mode, and core 3 in the constrained 7-Watt mode). The pipelines were restricted to the remaining cores above the Linux kernel (core 0) using Docker's `--cpuset-cpus` argument (cores 1-4 and cores 1-2, respectively). This strategy ensured separation between data generation, pipeline execution, and OS interruptions.

The latest stable releases of the language toolchains, which were compatible with the Ubuntu 22.04-based root file system, were used for all implementations: C++20 with GCC 15.2.0, Rust 1.97.1 and CPython 3.10.12. CPython is the default Python interpreter for many Linux distributions, and implements the runtime model, including the GIL and GC, that is evaluated in this report.

4.3. Deterministic Load Generator

To reliably compare the performance of the three implementations, a separate synthetic load generator, shown in Figure 2 (overleaf), was developed to create reproducible and deterministic simulated sensor data. This ensures that the behaviour of each implementation can be compared using the same baseline data, and that differences in performance can be attributed to the runtime models and backpressure policies, and not input variability.

The load generator produces three streams of data to shared memory buffers for consumption by the HAR pipelines: (1) an RGB video stream to simulate the camera, (2) a 3-axis inertial measurement stream to simulate the accelerometer, and (3) a 3-axis inertial measurement stream to simulate the gyroscope. Using shared memory allowed for low-latency communication, and for the generator to write data at a consistent rate. The shared memory buffers were implemented as fixed capacity ring buffers, allowing the generator to write data without being blocked by the pipeline.

The generated image data consisted of random noise. Each image was created as an array of RGB pixel values with dimensions of 1920x1080 to match the sensor data, and each pixel's red, green, and blue values were assigned random whole numbers in the range

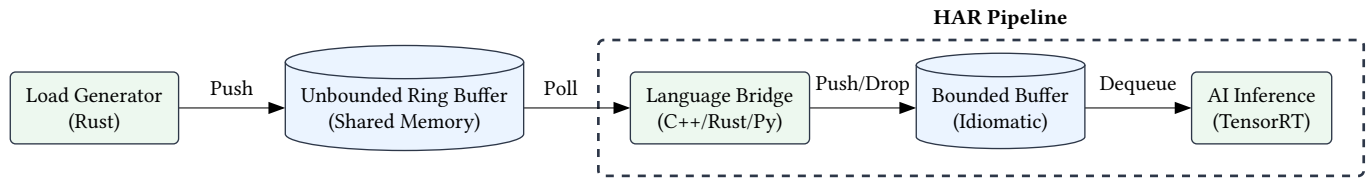


Figure 2: System architecture demonstrating communication between the separate Load Generator and the HAR Pipeline implementations.

[0, 255]. Similarly, the generated IMU data consisted of random floating-point numbers within the maximum hardware ranges of the BMI088 sensor ($\pm 24g$ for acceleration and $\pm 2000^\circ/s$ for angular rate).

A hard-coded seed for each sensor ($rgb = 42, accel = 43, gyro = 44$) was used to create the three deterministic data-streams to ensure the same generated data was fed into each implementation. To improve runtime efficiency and allow faster generation, the random synthetic data for each stream was pre-generated during program initialisation to avoid the overhead of continuous pseudo-random number generation. Each synthetic data pool was large enough for a continuously cycled temporal window of one second to allow variation in the data streams without exhausting the device’s available memory. Because the AI inference is only a repeatable workload to determine the performance of the runtime models, and prediction accuracy does not impact the evaluation, the generated data was not designed to be realistic. This kept the load generator implementation simple, does not introduce disk I/O bottlenecks, and reduced latency and overhead that could confound the results by contending with the pipeline for shared memory bandwidth or polluting shared caches.

To allow backpressure policies to be evaluated under varying load, the generator accepts a *load* parameter that dictates the speed at which data is produced by acting as a multiplier of the baseline sensor intervals. This multiplier is applied equally to all three data streams to ensure the ratio between the camera and the IMU data remains consistent. For example, a value of 1.0 produces data at the same rate as the sensors: 30 FPS for the camera (1 frame every 33.3 ms), and 1.6 kHz and 2.0 kHz for the accelerometer and gyroscope respectively (0.625 ms and 0.5 ms intervals respectively). A value of 2.0 produces data twice as fast, 0.5 produces data at half the speed, and so on.

The load generator was written in Rust to take advantage of its performance and memory safety guarantees. For maximum timing accuracy, `nix::time::clock_nanosleep` was used to implement the timing of the data generation. `CLOCK_MONOTONIC` was used for high resolution timing, and Network Time Protocol (NTP) synchronisation was disabled using `timedatectl set-ntp false` to prevent the system clock from being adjusted during the experiments.

The deterministic load generator and HAR pipelines were designed to allow seamless substitution of the generator with a physical hardware harness, using shared memory (`/dev/shm`) ring buffers as the communication boundary. A physical hardware test was initially planned. However, damage to the Jetson Orin Nano Developer Kit’s MIPI CSI-2 Zero Insertion Force (ZIF) connector during assembly prevented the connection of the Waveshare IMX219-160 Camera Module. Investigation revealed that these ZIF connectors are notoriously fragile, further validating the need for a deterministic generator to execute the high-stress evaluations required for this dissertation’s research.

4.4. Backpressure Policies

Queue management and backpressure policies are implemented in each language-specific runtime model. In a typical backpressure implementation, when a *consumer* is saturated (i.e. the buffer is full), the active backpressure policy is triggered to slow the flow of data from the *producer* to prevent unbounded memory demand and system instability.

The backpressure policies were implemented in the pipelines using two buffers per data stream: (1) an unbounded ring *producer buffer* in shared memory for the load generator to write data into, allowing it to produce data at a consistent rate, and (2) a *consumer buffer* for the pipelines to read data from for processing, with a fixed capacity to trigger the backpressure policy when full.

Using Little’s Law ($L = \lambda W$) [30], the capacity of each consumer buffer was determined by multiplying the *target* baseline throughput (λ) by the maximum deadline (W) for processing an event (Figure 3). The target throughput is based on the *native* generation rate of the physical sensors (i.e. a load multiplier of 1.0), allowing the pipelines to be tested under simulated system stress relative to the baseline speed of the physical sensors. As stated in Section 1.2, a maximum deadline of 100 ms was selected based on Xue et al. [1], resulting in a capacity of 3 for the RGB stream (30 FPS), 160 for the accelerometer stream (1.6 kHz), and 200 for the gyroscope stream (2.0 kHz).

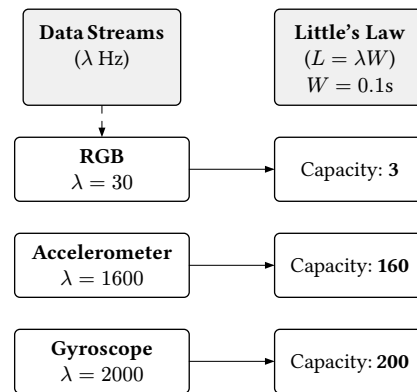


Figure 3: Application of Little’s Law to derive the bounded queue capacities from the data stream rates and the maximum latency deadline of 100 ms.

The bounded buffers were implemented as fixed-capacity circular queues using contiguous arrays in each language (`std::vector` in C++, `Vec` in Rust, and a `list` in Python). Standard mutexes were employed to guarantee multi-threading safety during enqueue and dequeue operations.

Backpressure is implemented only in the pipeline on the consumer buffer, forcing each language runtime model to handle concurrency, memory allocation, and scheduling within realistic constraints and allowing RQ2 to be evaluated. A *bridge* in the pipeline is responsible for copying data from the producer buffer to the consumer buffer, and for triggering the backpressure policy when the consumer buffer is full.

Five backpressure and load shedding policies were implemented to manage queue saturation when the consumer buffer is full:

Policies that attempt to preserve all data (Flow Control):

- **Bounded Queue:** Blocks the pipeline’s bridge thread from ingesting new frames until space becomes available in the consumer buffer.
- **Exponential Backoff:** Waits a short time before retrying to insert the data, with the wait time increasing each time by a configurable factor until a maximum wait time is reached (after which the data is dropped).

Policies that intentionally drop data (Load Shedding):

- **Drop Oldest:** Drops the oldest data in the consumer buffer to make room for new data.
- **Drop Newest:** Drops incoming data when the buffer is full.

Policies that dynamically drop data while preserving temporal continuity:

- **Adaptive Decimation:** Dynamically downsamples the data stream (i.e. queueing only every n -th frame) to reduce pressure on the consumer buffer while preserving the temporal continuity of the data. As the consumer buffer fills, the decimation factor is increased to reduce the number of frames being queued. Similarly, as the consumer buffer empties, the decimation factor is decreased. If the queue reaches full saturation, the oldest frame is dropped to make room for the newest frame.

Bounded queue and exponential backoff are both flow-control policies. Instead of actively dropping data, they stall the pipeline's bridge thread when the consumer buffer is full. However, because the upstream load generator continues to write to the fixed-capacity shared memory ring buffer, stalling eventually causes unread frames to be overwritten (lapped). This guarantees a maximum memory footprint, but results in data loss if the stall exceeds the temporal capacity of the ring buffer. Conversely, drop-oldest, drop-newest, and adaptive decimation are load-shedding policies that discard data (as visualised in Figure 4).

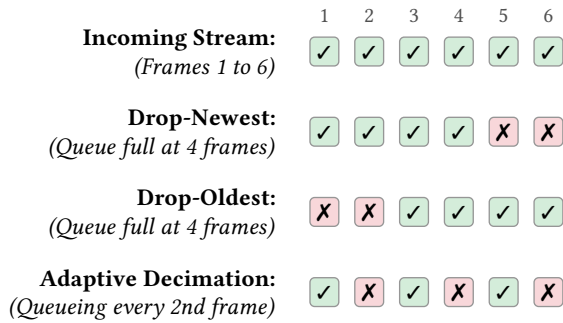


Figure 4: Load shedding policies on a stream of frames. Green check-marked (✓) blocks represent preserved data, red cross-marked (✗) blocks represent dropped data.

The exponential backoff policy was configured with an initial wait time of 1 ms to give sufficient time for the scheduler to yield to the inference thread, providing an opportunity for space to become available in the consumer buffer. The wait time is doubled upon each retry, up to an accumulated maximum of 33.3 ms before the frame is dropped. This maximum wait time was derived from the generation interval of the 30 Hz RGB anchor stream. If exponential backoff were to wait longer then it would cascade delays to the next late-fusion window. Therefore, dropping the stalled frame effectively resets the pipeline, providing an opportunity to recover and the next frame to be ingested in time to meet the 100 ms latency deadline.

The adaptive decimation policy was configured to activate at 80% of each stream's consumer buffer capacity (i.e. 2 frames for RGB, 128 for accelerometer, and 160 for gyroscope). This provides 20% of queue capacity for the algorithm to dynamically scale how much load is shed before the queue is saturated. At 80% capacity, the pipeline is struggling to keep up with the rate of generation, and so 50% of the frames are dropped (i.e. every second frame is queued) to allow the pipeline to recover. If the consumer buffer continues to fill, the decimation factor is linearly scaled until it reaches a maximum of 90% drop-rate (i.e. only every 10th frame is queued by overwriting the oldest frame) at full saturation. Should the consumer buffer remain at full saturation, the oldest frame is dropped to make room for the newest frame, ensuring that the most recent data is retained for inference.

To ensure the runtime models were evaluated under sustained stress, the saturation threshold was determined by increasing the *load* multiplier until at least one consumer buffer reached capacity, ensuring the backpressure mechanism was continuously engaged across all policies. Because the languages differ in baseline performance, a separate, fixed load multiplier was determined for each language. This normalised the level of system stress, ensuring that all five backpressure policies within a given language were evaluated against a language-appropriate rate of ingestion.

4.5. False Sharing

Modern CPU architectures contain multiple caches for each core to significantly reduce memory access latency. These are designated a level (e.g. L1, L2) to indicate how far away they are physically from the CPU core, and typically increase in size and latency as their level increases. Each cache is split into multiple *cache lines*, where the data is stored and can be written or read significantly faster than from main memory. However, because the low-level cache lines are not shared between cores, if a process on one core updates data in a cache line, then processes on other cores that need to read that data must update their own cache lines by fetching the refreshed data from higher-level shared caches or main memory.

False sharing occurs when an update to data held in a cache line causes other cores to invalidate their own cache lines, and the data being read is otherwise unrelated to the data being written. For example, if a 32-byte structure is stored in RAM at address range 0x1000–0x1020, a CPU with 64-byte cache lines will store 64 bytes of data in its cache line covering the address range 0x1000–0x1040, even though the last 32 bytes are unrelated to the original structure. If those adjacent 32 bytes are updated, it will trigger a refresh of the entire 64-byte cache line, invalidating the first 32 bytes and forcing a read penalty.

Because the shared memory buffers are held in contiguous memory, false sharing needed to be mitigated to prevent cache line invalidation of the header when adjacent data frames were updated. This was achieved by using language-specific techniques to pad the header to exactly 64 bytes in size and align its starting address to a 64-byte boundary, ensuring it would be cached in isolation.

While the shared memory buffers are bound to one-second cycles, and consequently false sharing would occur only twice per second if 64-byte alignment were not enforced (once for the timestamp and once for the payload), ensuring cache isolation of the header is best engineering practice and prevents potential system degradation caused by future modifications to the implementation.

4.6. Memory Ordering

Traditional mutexes force a thread to yield to the system kernel. This introduces latency and context-switching jitter, which may cause the generator and pipelines to violate their timing constraints at the Inter-Process Communication (IPC) boundary. To avoid this, wait-free spin-loops were used, which required the use of atomic *memory ordering*.

The load generator updates the unbounded ring buffer's `seq_num` to inform the pipeline that a new data frame has been written, and the pipeline spin-loops and reads the new data as soon as it sees the `seq_num` updated. Without memory ordering, the CPU may choose to update these apparently unrelated variables “out of order” from what the code specifies. This would be catastrophic, as the pipeline may read the stale data before the new data is committed. To prevent this, the `seq_num` was declared as an atomic variable, and release memory ordering was used when the generator updated it, informing the CPU that all previous writes to any variable must be committed before the `seq_num` is updated. Conversely, when the pipeline reads the `seq_num`, it uses acquire memory ordering to inform the CPU that it must not speculatively read any other variables before the `seq_num` is read. This simple “fence” guarantees that

the memory ordering is correct, and that the apparently unrelated `seq_num` and data frame are read in the correct order.

4.7. Zero-Allocation

To reduce memory churn and the latency jitter that may be introduced by high-frequency dynamic memory allocation (as well as GC pauses in Python) a zero-allocation approach was used. This was achieved by ensuring all necessary memory (e.g. the bounded queue) was pre-allocated during initialisation, and lightweight data structures (e.g. the telemetry Epoch) were continuously reused rather than reallocated.

A buffer was pre-allocated in each inference thread to copy the payload from every frame in the temporal window, providing a contiguous data source for the TensorRT engine without utilising dynamic memory allocation. This was necessary because the shared memory buffer operates as a ring, and so a temporal window may wrap around the physical memory, causing later frames to precede earlier frames. Furthermore, each frame contains a timestamp before the payload, causing the data to be strided. The contiguous buffer was bound to the inference execution context (using `Ort::IOBinding` for C++, `ort::value::Tensor` for Rust, and `onnxruntime.IOBinding` for Python) to prevent the TensorRT engine from dynamically allocating memory or internally copying data during inference.

4.8. AI Model Generation

Hardware-agnostic `.onnx` AI model files were generated offline using PyTorch 2.13.0. Dynamic axes (i.e. allowing the inference data to be of variable sizes) were forbidden to ensure that the TensorRT engine would not dynamically allocate memory during inference, instead allocating memory once upon startup, thus improving performance and preserving the zero-allocation approach. These models were then transferred to the Jetson Orin Nano and saved to hardware-specific `_epctx.onnx` (Execution Provider Context) files using ONNX Runtime 1.24.0 before pipeline evaluation commenced. These files ensure that the models do not need to be re-optimised at startup for each evaluation. While C++ and Python were able to share the same context files, Rust uses a newer C-API and thus was required to cache its own versions to disk. However, due to the shared `.onnx` model files, both sets used identical parameters and optimisations, ensuring functional equivalence across all three implementations.

4.9. Profiling and Metrics

4.9.1. Late Fusion

A Multi-Producer Single-Consumer (MPSC) pattern was used to drive the late fusion, where the inference threads all pushed their results to a single fusion thread for processing, and the fusion execution was tied to the 30 Hz RGB stream. Anchoring on the slowest, most computationally expensive stream prevented both redundant fusion executions, and the fusion thread from being bottlenecked by the faster IMU streams. Consequently, the IMU streams were downsampled using fixed window sizes to match the 30 Hz RGB

stream (Figure 5), ensuring that the IMU inference was executed, and the results were injected into the MPSC channel, only when the window was full. Zero-Order Hold was used to pair the RGB and IMU inference results, where the most recent IMU inference result was held until the next RGB inference result was available.

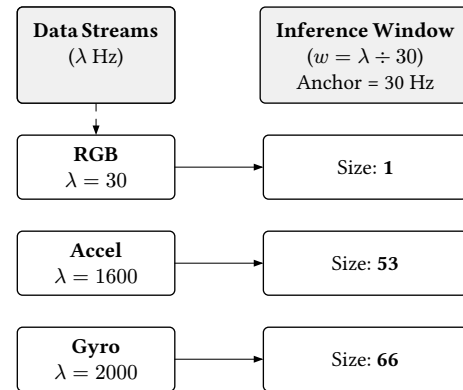


Figure 5: Derivation of the inference window sizes for the IMU streams to match the 30 Hz RGB stream, using Zero-Order Hold to pair the inference results.

4.9.2. Latency Breakdown

To measure latency of the pipelines, the `CLOCK_MONOTONIC` clock was used to capture high-resolution timestamps at key points as the frames flowed through the pipeline. NTP synchronisation was disabled to prevent the system clock from being adjusted. The following six timestamps, as visualised in Figure 6, were captured for each event:

1. `generated_ts` when the generator pushes to the unbounded ring buffer
2. `bridged_ts` when the bridge pushes to the bounded buffer
3. `pipeline_in_ts` when the inference thread pulls the frame from the bounded buffer
4. `pipeline_out_ts` when the ONNX Runtime completes inference and the result is pushed to the Multi-Producer Single-Consumer (MPSC) channel
5. `fusion_in_ts` when the late-fusion thread pulls the inference results from the MPSC channel
6. `fusion_out_ts` when late-fusion execution completes and the pipeline produces the final output

To prevent Coordinated Omission, as identified in Section 3.4, the load generator is decoupled from the pipelines. By ensuring that it pushes to an unbounded ring buffer, it is never blocked when the system is stalled, thus guaranteeing that `generated_ts` allows queueing delays and tail-latency to be accurately captured.

These timestamps provide five key latency measurements: *Unbounded Queue Wait* (`bridged_ts` – `generated_ts`), *Bounded Queue Wait* (`pipeline_in_ts` – `bridged_ts`), *Inference Execution* (`pipeline_out_ts` – `pipeline_in_ts`), *MPSC Wait* (`fusion_in_ts` –

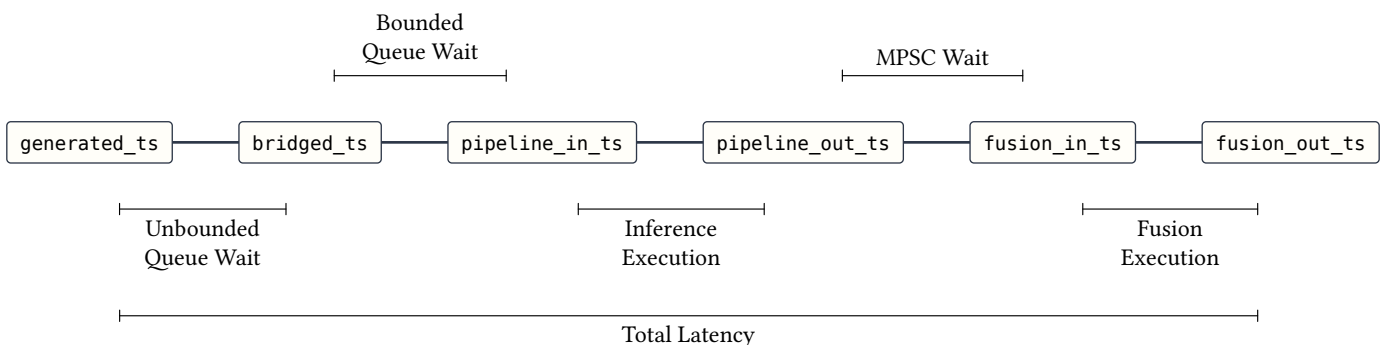


Figure 6: Timeline of the six timestamps captured for each event as the frame flows through the pipeline.

pipeline_out_ts), and *Fusion Execution* (fusion_out_ts – fusion_in_ts). Additionally, *Total Latency* (fusion_out_ts – generated_ts) was calculated to capture the end-to-end processing time.

These measurements provide the necessary granularity to measure each runtime model's latency, and to identify bottlenecks and trade-offs under load and backpressure. Because the late-fusion execution is anchored to the 30 Hz RGB stream, the RGB telemetry was used for end-to-end latency analysis, as the IMU frames must wait for late-fusion synchronisation. However, to determine system capacity, dropped and lapped frames are evaluated across all three streams.

To retain temporal information about how latency changes over time and correlates with runtime model behaviour and backpressure events, a triple-buffering approach was used (see Figure 7). The pipeline thread (the *writer*) populated an active Epoch object containing the latency histograms, memory counters, and dropped frames counter. At one-second intervals, a clean Epoch was pulled from a channel using wait-free message passing, and the populated Epoch was pushed to another channel. Concurrently, a lightweight background telemetry thread (the *reader*) pulled the populated Epoch messages from the second channel and saved the counters and the p50, p95, p99, p99.9, and maximum latency values into a CSV file, then reset the Epoch and pushed it back to the first channel for reuse. A third Epoch was kept idle in the first channel ready to be swapped in as the new active buffer, preventing any blocking of the pipeline thread if the telemetry thread is delayed (e.g. by I/O stalls).

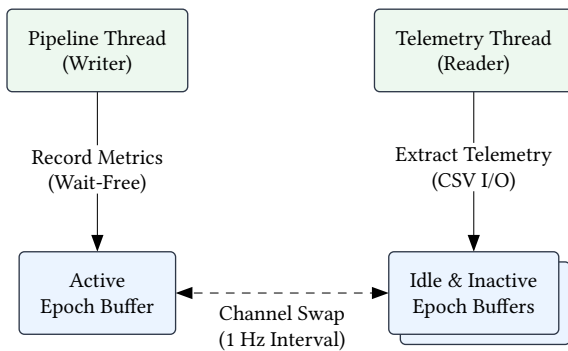


Figure 7: Triple-buffering approach to cleanly capture the telemetry epochs without blocking the pipeline thread during stalls.

High Dynamic Range (HDR) Histograms [31] were used to aggregate the latency distributions, preventing memory allocation from polluting the latency measurements that would occur if the measurements were stored in standard data structures (e.g. vectors or lists).

4.9.3. Event Synchronisation

To ensure that identical event streams were processed by each implementation, without introducing startup jitter or missing initial events, an atomic variable (pipeline_stage) was integrated into each shared memory buffer header. This variable was initialised to 0 (WAITING) by the generator, and set to a value of 1 (READY) by the pipeline bridges once they were fully initialised and ready to receive data. The load generator spin-waited until all three pipelines were ready before starting to push data.

The generator updated the pipeline_stage variables to 2 (FINISHED) as each event stream was completed. Each pipeline bridge processed all remaining valid frames from the shared memory buffer, and then used a *poison pill* technique to signal the end of the stream. A special frame containing a maximum sequence number (UINT64_MAX, u64::MAX, or (1 << 64) - 1) was injected into the bounded queue, which initiated a graceful shutdown of the pipeline by all threads after any pending frames were processed.

4.9.4. Memory Churn (C++ and Rust)

To measure the rate of memory churn in C++ and Rust (RQ3), the global memory allocation and deallocation functions were overridden to capture memory allocation metrics, without relying on third-party profiling tools that may introduce additional overhead and confound the results. In C++, the operator new and operator delete functions were overridden, and in Rust a custom memory allocator was implemented as the standard library's default by using the #[global_allocator] attribute.

The telemetry thread concurrently captured the memory allocation metrics during the same intervals as the latency measurements, allowing for correlation between memory churn under load and backpressure events. Three atomic operations (std::atomic<size_t> in C++, and AtomicUsize in Rust) were used to capture the total number of allocations, total bytes allocated, and total bytes freed. Relaxed memory ordering was used to prevent the observer effect from confounding the results by introducing additional latency. Though this may introduce nanosecond-level "read skew" when the telemetry thread reads the counters (i.e. the independent metrics are read slightly out of sync with each other), the telemetry thread only reads these metrics once a second, which renders this comparatively insignificant temporal drift statistically irrelevant.

4.9.5. GC Pressure (Python)

Python uses a GC to manage memory, which can introduce non-deterministic tail-latency GC pauses (also known as "stop-the-world" events) when run. Using tracemalloc from the standard library would introduce additional overhead and confound the results, as it introduces tracing for every memory allocation event. Instead, the GC's built-in callbacks hook was utilised to capture the start and end time of each GC event (using CLOCK_MONOTONIC) to calculate the duration of each pause.

To prevent memory allocation within the callback function, a triple-buffering approach was used, similar to the latency measurements, where the callback function writes the GC pause durations to an active Epoch without blocking. The background telemetry thread then extracts the total accumulated GC pause duration at the same time as the latency measurements, allowing correlation between GC pause durations and runtime model events.

4.9.6. Memory Fragmentation

Repeated allocation and deallocation of memory can lead to fragmentation, where free memory is only available in small non-contiguous blocks. As shown in Figure 8, this can cause memory to be exhausted even when the total free memory is sufficient, as contiguous blocks larger than the fragmented sizes are not available, leading to Out-Of-Memory (OOM) errors.

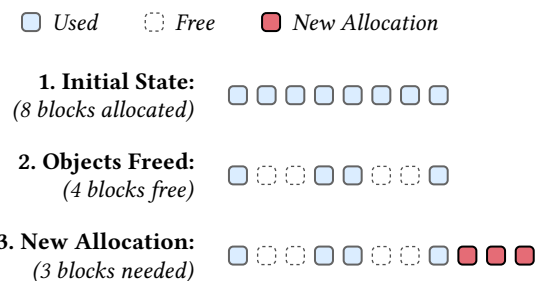


Figure 8: Visualisation of memory fragmentation. Though the total free memory (4 blocks) is sufficient for the new allocation (3 blocks), the allocator must expand the heap due to the lack of contiguous space, increasing the RSS.

To ensure a fair comparison, all three implementations use the Linux interface /proc/self/statm to capture the Resident Set Size (RSS) from the background telemetry thread. The RSS provides the total amount of memory currently allocated to the process, including

fragmented memory and that allocated by third-party libraries (e.g. ONNX Runtime).

In addition, the C++ and Rust implementations used `mallinfo2()` to capture the `fordblks` field, which provides the total size of memory allocated by the process that is currently free, providing insight into the amount of fragmented memory that is allocated but not currently in use.

4.9.7. Power Modes and Cooling

When the Jetson Orin Nano reaches the `hot_surface_alert` trip point of 74°C, it automatically engages the cooling fan. This is a hardware-level protection that cannot be disabled. If the fan fails to provide enough cooling, the device utilises reactive software thermal management (DVFS) that constantly polls the temperature and throttles the performance of the high-power components (e.g. CPU and GPU) when the device exceeds operating temperature threshold at 99°C (see Appendix C).

The Jetson was configured to use the unconstrained power mode (MAXN_SUPER) using `nvpmodel -m 2` (see Appendix B), and maximum clock overrides were enabled using `jetson_clocks`. This allows the device to operate at its maximum performance. Kernel console logging was disabled (using `dmesg -n 1`) to prevent I/O interrupts from affecting the measurements. Temperature, power draw, and clock frequencies were recorded to a `.log` file using the `tegrastats` utility, and converted to a CSV file for analysis after the evaluation using `awk`.

All evaluations were conducted in an environment with an ambient room temperature ranging between 21.4°C and 26.2°C. Prior to commencing the evaluation suite, the Jetson Orin Nano was rebooted and allowed to idle for ten minutes to stabilise. A target baseline temperature of 60°C was selected for all evaluations. Experimental testing determined that lower thresholds (e.g. 55°C) could not be reliably or rapidly achieved between pipeline executions due to residual heat and insufficient ambient dissipation.

To enforce the baseline temperature, user-space active cooling (`nvfancontrol`) was disabled, and the fan was stopped by echoing 0 to `/sys/class/hwmon/hwmon0/pwm1`. The Jetson was then kept busy (`cat /dev/urandom > /dev/null`) for periods of 2 seconds at a time, until the baseline temperature was reached or exceeded. The device was then allowed to idle with the fan speed set to 100% (255) until the baseline temperature was reached or fell short of, at which point the fan was stopped. This was repeated as necessary until the baseline temperature was exactly reached before the evaluation started.

Initial attempts to engage DVFS were unsuccessful, as the Jetson's fan was able to cool the device fast enough to prevent the throttling temperature from being reached. Therefore, a second evaluation was performed after restricting the device to the 7-Watt power mode (using `nvpmodel -m 3`), and maximum clock overrides were disabled (`jetson_clocks --restore`), forcing the device to throttle the CPU and GPU to adhere to the power budget, thus allowing an evaluation and comparison of the pipeline under constrained conditions.

4.9.8. Statistical Analysis

Latency measurements have no theoretical maximum, but are inherently bounded by a minimum value of zero. This typically results in non-normal distributions which are heavily skewed to the right, with long tails and outliers [18], requiring non-parametric methods for statistical analysis. The outliers are evidence of backpressure events and runtime model pauses (e.g. Garbage Collection in Python), necessary for benchmarking and implementation comparisons, and consequently were not removed.

Steady-state measurements were isolated from the cold-start artefacts to prevent skewing. A fixed 10-second warm-up boundary was used across all evaluations. Visual analysis of the initial logs

confirmed that this was more than sufficient for third-party initialisation to complete and for latency to stabilise.

Descriptive statistics were required to summarise the performance. Because the mean is sensitive to the aforementioned outliers and skewed distributions, it would not provide an accurate measure of central tendency. Instead, the median (p50) was used to represent typical performance. The p95, p99, p99.9, and maximum latency values were used to describe the worst-case performance measurements.

Beyond descriptive percentiles, statistical testing was necessary to evaluate the implementations. The Kruskal-Wallis H test [32], a non-parametric method that is robust to non-normal distributions and outliers, was used to compare the latency and throughput distributions from the three runtime models. Because of the large sample sizes, the epsilon-squared (ϵ^2) effect size [33] was calculated to determine how much of the latency variance could be attributed to the runtime model. Dunn's test [34] was then used for post-hoc analysis to identify which implementations differed significantly, with a Bonferroni correction [35] to control the error rate and to prevent false positives.

Finally, to investigate the root causes of the observed performance bottlenecks, Spearman's rank correlation coefficient (ρ) [36] was used to identify statistically significant correlations between latency and system events (e.g. GC pauses, dynamic memory allocations, and hardware power constraints). Unlike Pearson's correlation coefficient (r) [37], Spearman's ρ considers the rank of the data rather than the raw values, making it robust to non-normal distributions and extreme outliers. Furthermore, Spearman's ρ can capture monotonic relationships that are not strictly linear (i.e. relationships that consistently increase or decrease, but not necessarily at a constant rate) [38], allowing correlations to be identified even when exponential degradation occurs.

4.9.9. Code Verbosity and Complexity

While this report's primary analysis is focused on comparing the performance of the C++, Rust, and Python runtime models, language selection is often influenced by development, maintenance, and testing overhead [39]. To evaluate the trade-off between runtime efficiency and implementation verbosity, a supplemental static code analysis was performed to compare the pipeline implementations.

Lizard [40] is a code complexity analyser that supports C++, Rust, and Python. It was utilised to determine: (1) the Non-commenting Lines Of Code (NLOC), quantifying how verbose each implementation is, and (2) the Cyclomatic Complexity Number (CCN) [41], quantifying the number of linearly independent paths that exist in each implementation's source code. By measuring the NLOC and CCN of the functionally identical backpressure and concurrency implementations in C++, Rust, and Python, this analysis provides a partial insight into each runtime model's development lifecycle overhead.

4.10. Methodological Limitations

4.10.1. Memory Churn Asymmetry

An asymmetry exists in the measurement of dynamic memory allocation across the implementations. When overriding operator `new` and operator `delete` in C++, allocations made by third-party headers (e.g. `moodycamel::BlockingConcurrentQueue`) are captured, but allocations made internally by pre-compiled shared libraries (e.g. `libonnxruntime.so`) are not. Similarly, in Rust, allocations made by wrapper crates (e.g. `ort`) are captured, but those in the underlying pre-compiled libraries are not.

Furthermore, while C++, Rust, and Python all utilise ONNX Runtime's C-API, the Rust wrapper crate uses a newer version than that used by C++ and Python. Consequently, while all implementa-

tions use similar data structures to serialise data across the Foreign Function Interface (FFI) boundary, there may be unmeasurable differences in the memory management of the underlying C-API. An asymmetry also exists in the capture of memory allocation within Python's third-party C-extension bindings (e.g. `onnxruntime`), which do not use Python's memory manager and thus are not visible to the telemetry thread. While this asymmetry is a limitation when comparing dynamic memory churn within third-party libraries, the methodology mitigates this by recording the RSS that captures all memory demand regardless of its origin or allocator.

4.10.2. Read-Tearing and Misalignment

Because the shared memory buffer is circular and unbounded, there is a risk that the producer overwrites a frame while the consumer is reading it, leading to misalignment or read-tearing of the sensor data. This would be unacceptable in a production environment, requiring a separate *memory arena* to store the sensor data and guarantee that the producer does not overwrite a frame until the consumer has released it. However, implementing such a solution would effectively bound the load generator, causing it to block when the pipeline is saturated, thus reducing the system pressure and hiding the queueing delays that this report aims to measure.

As the HAR pipeline serves only as a testbed, and the accuracy of the AI inference does not form a part of the evaluation, the system accepts the possibility of read-tearing to preserve the unbounded nature of the load generator. This is minimised by adopting a 1-second ring buffer paired with the 100 ms latency deadline, building a 900 ms temporal safety margin into the architecture, which is sufficient to ensure that read-tearing would only be possible in the event of a major system stall (e.g. an extreme GC "stop-the-world" pause or severe thermal throttling event).

4.10.3. Global Metrics Desynchronisation

In C++ and Rust, the dynamic memory allocation metrics were captured at a global level by overriding the system allocators. Similarly, the RSS and Python's GC pauses are process-wide metrics, rather than on a per-thread or per-sensor basis.

Because the three concurrent telemetry threads operated on independent 1-second epochs, these global metrics were slightly desynchronised, resulting in minor recording variations between the three sensor logs. For this reason, only the RGB telemetry log was used to analyse the process-wide memory and GC metrics. While this introduces a small desynchronisation between the global metrics and the IMU latency measurements, the 1-second epoch is sufficiently long to ensure that the nanosecond-level desynchronisation is statistically irrelevant.

4.10.4. Temporal Alignment of Late-Fusion

The inference window capture-count introduces a temporal misalignment between the IMU and RGB streams. Because the inference threads construct the temporal windows by counting the frames, rather than by using timestamp deltas, aggressive load-shedding effectively stretches the window, causing misalignment during late-fusion. Furthermore, because the IMU sensor speeds (1600 Hz and 2000 Hz) are not wholly divisible by the anchoring RGB sensor speed (30 Hz), it is recognised that temporal alignment drift is inevitable even in the absence of load-shedding.

This would need to be mitigated in a production environment by using a timestamp delta to bound the temporal window. However, this would introduce a confounder to this report's evaluation, as the workload would fluctuate according to language-specific overhead (such as GC pauses in Python). Therefore, an inference counter was required to allow a true comparison of the overhead of the three runtime models under identical load conditions.

4.10.5. Temporal Alignment of Telemetry

Because `tegrastats` was executed as a separate process, its sampling intervals were not fully synchronised with the pipeline implementations. Therefore UNIX timestamps (`CLOCK_REALTIME`) were recorded by both systems. Nearest-neighbour interpolation was used during data aggregation to align the `tegrastats` telemetry with the pipeline's latency and memory metrics, allowing for the correlation of thermal events with performance degradation. This introduces a maximum temporal misalignment of ± 500 ms. Because thermal and throttling state changes occur over seconds rather than milliseconds, this sub-second uncertainty is accepted as a necessary limitation not expected to significantly confound the results.

4.10.6. Static Complexity Analysis

Regarding the code verbosity and complexity analysis, metrics such as NLOC and CCN only consider the static source code, and do not consider the learning curve or cognitive complexity associated with each language (e.g. C++'s manual memory management, Rust's borrow checker and ownership model, or Python's dynamic typing). These can all significantly influence the development lifecycle overhead, and therefore NLOC and CCN should be interpreted as partial measurements of the engineering cost of language selection.

5. Implementation

5.1. System Architecture Overview

To evaluate the language runtime models of C++, Rust, and Python, a functionally identical multi-threaded HAR pipeline was implemented across all three languages. As shown in the end-to-end diagram in Figure 9 (overleaf), individual threads are responsible for each stage (bridge, inference, late-fusion, and telemetry). Further, a separate chain of threads is spawned for each unbounded ring buffer (RGB, accelerometer, and gyroscope), with the exception of the late-fusion thread, which uses the MPSC pattern to fuse the inference results from all three streams into one prediction.

Each spawned thread chain begins at the **bridge thread**, which spin-waits on an unbounded shared memory (`/dev/shm`) ring buffer populated by an external process, defining the Inter-Process Communication (IPC) boundary. The bridge thread attempts to add ingested frames into a bounded queue, applying the configured backpressure policy (e.g. Adaptive Decimation, Drop Oldest) if the queue is full. The **inference thread** pulls frames from the bounded queue to create temporal event windows, execute the ONNX model, and push the result to the MPSC channel. The **late-fusion thread** consumes from the MPSC channel and anchors execution of its own ONNX model to the 30 Hz RGB stream, before finally passing the frames to the individual **telemetry threads** for persistence of the telemetry metrics.

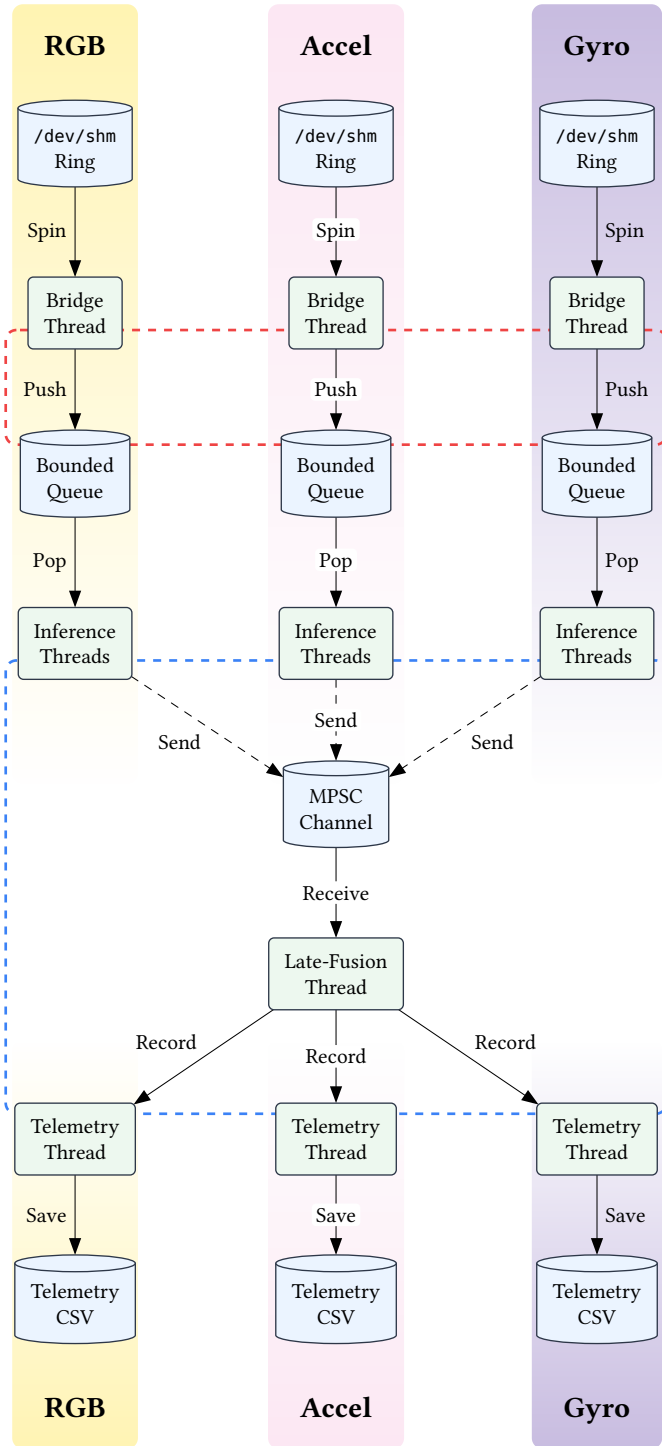


Figure 9: End-to-end flow demonstrating data ingestion, backpressure application, MPSC synchronisation, late-fusion, and telemetry capture. The red dashed region denotes the backpressure application boundary, while the blue dashed region highlights the late-fusion execution anchored to the 30 Hz RGB stream.

5.2. The Inter-Process Communication (IPC) Boundary

Shared memory (/dev/shm) was employed for the unbounded ring buffer, allowing IPC between the external process and the pipeline implementations. As shown in Figure 10, the buffer's header was sized to 64 bytes with additional padding to match the size of the Jetson Orin Nano's Cortex-A78AE L1 CPU cache line [23], allowing it to remain in a single, isolated cache line without the risk of false sharing when the adjacent raw frame data was modified by the producer. In C++ and Rust, this alignment was achieved using simple compiler directives (`alignas(64)` and `#[repr(C, align(64))]`, respectively). However, Python required manual memory mapping via `ctypes.Structure` inheritance to define the byte widths of the fields (`c_uint32`, `c_uint64`), as shown in Listing 1.

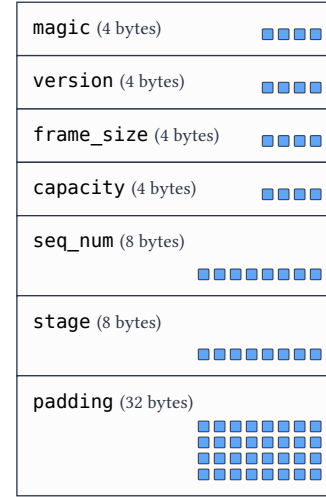


Figure 10: 64-byte aligned IPC memory header layout. Padding ensures the atomic fields consume exactly one CPU cache line, preventing false sharing.

```
struct alignas(64) ShmHeader {
    uint32_t magic;
    // ... omitted ...
    std::atomic<uint64_t> seq_num;
    std::atomic<uint64_t> pipeline_stage;
};
```

```
#[repr(C, align(64))]
struct ShmHeader {
    pub magic: u32,
    // .. omitted ...
    pub seq_num: AtomicU64,
    pub pipeline_stage: AtomicU64,
}
```

```
class ShmHeader(ctypes.Structure):
    _fields_ = [
        ('magic', ctypes.c_uint32),
        # ... omitted ...
        ('seq_num', ctypes.c_uint64),
        ('pipeline_stage', ctypes.c_uint64),
        ('padding', ctypes.c_uint8 * 32),
    ]
```

Listing 1: IPC Boundary structure definitions. C++ (top) and Rust (middle) use compiler directives for 64-byte alignment. Python (bottom) requires manual padding.

Mapping the shared memory data to the process's virtual memory address space provided zero-copy efficiency at the IPC boundary. While this was automatically handled in Python when instantiating the `multiprocessing.shared_memory.SharedMemory` class, C++ and Rust both required manual invocations of the `mmap` native UNIX system function. However, whereas the raw memory pointers in the compiled languages could be cast directly to the required structure with no overhead, Python transparently converts the raw bytes into native Python objects when accessing the fields of the `ctypes.Structure` [42]. This adds CPU overhead that is not present in the compiled languages. Furthermore, unlike the compiled languages, Python's `ctype` structures lack atomic types with explicit memory ordering. Therefore erecting the "fence" (see Section 4.6) using `acquire` memory ordering was not possible, and the data from the unbounded buffer was read without any guarantee of memory ordering.

5.3. Adaptive Decimation Backpressure Policy

When the bounded queue is full, the bridge thread uses a configured backpressure policy (see Section 4.4). One policy available to the bridge, Adaptive Decimation, sheds load at a dynamic rate, while trying to retain temporal continuity, by downsampling the stream

at an increasing rate as the queue enters a configured “danger zone” (e.g. 80% of queue capacity) *before* the queue is full.

As shown in Figure 11, after reading a frame from the unbounded ring buffer, the bridge thread determines if the length of the bounded queue is within the danger zone. If so, a decimation ratio value is calculated based on a linear scale between the minimum and maximum ratios, and the current depth of the queue within the danger zone. A counter is incremented for every frame, and frames are only pushed to the bounded queue when this counter is wholly divisible by the ratio. If the push is unsuccessful, the oldest frame is overwritten.

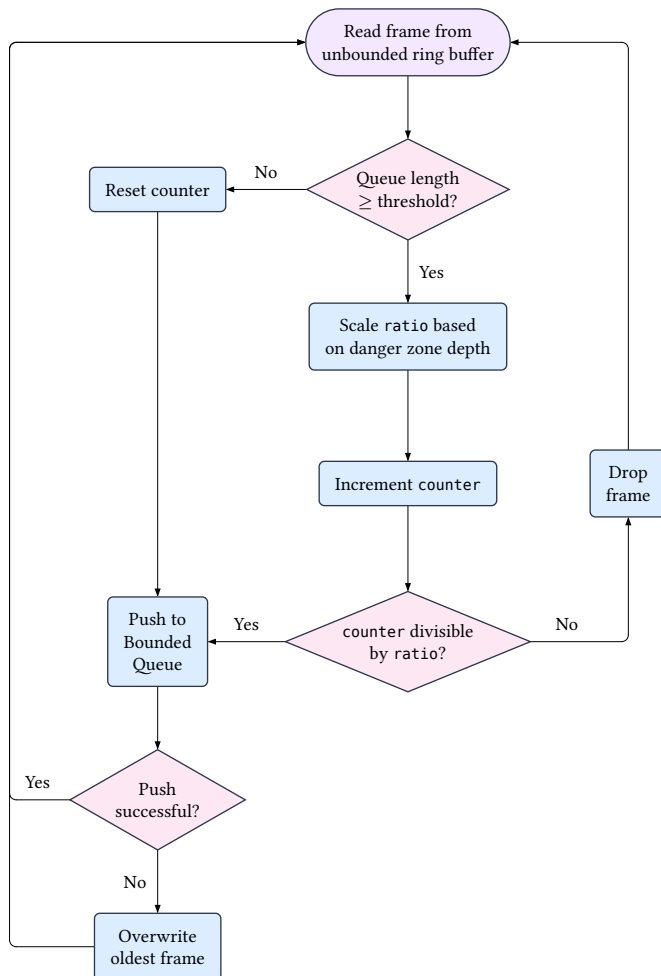


Figure 11: Flowchart detailing the Adaptive Decimation backpressure policy. The algorithm dynamically scales load-shedding based on queue saturation while preserving temporal continuity.

When implementing this algorithm across the three language runtime models, a difference was found between the statically typed languages (C++ and Rust) and the dynamically typed Python. When calculating the ratio, the statically typed languages naturally truncated the result of the division to an integer. However, in Python, standard division (/) returns a floating-point value, which caused virtually all frames to be incorrectly dropped when performing the modulo operation. This subtle difference highlighted the risk of dynamic typing, and resolving this required the addition of a second forward-slash character to apply the floor-division operator (//).

5.4. Wait-Free Ingestion

To avoid introducing latency and context-switching overhead, it was necessary to spin-loop while waiting for new frames at the IPC boundary, without yielding to the system kernel through the use of OS-level mutexes or sleep instructions. In C++, a small preprocessor macro was implemented to execute the ARM64 `yield` assembly instruction, while `std::hint::spin_loop()` was used in Rust. These *micro-architecture* hints instruct the CPU to temporarily pause

speculative execution (when the CPU predicts the next instruction and executes it ahead of time, before the previous instruction has completed) and to throttle the CPU for a few clock cycles, reducing power usage and generated heat.

Conversely, there is no micro-architectural hint available in Python to yield to the CPU. Instead, an empty pass loop was utilised (see Listing 2). This forces the thread to spin tightly without yielding to the CPU, holding the GIL and starving the other threads of execution time.

```
#if defined(__x86_64__) || defined(_M_X64)
#include <immintrin.h>
inline void spin_loop() { _mm_pause(); }
#elif defined(__aarch64__)
inline void spin_loop() {
    __asm__ volatile("yield" ::: "memory");
}
#else
inline void spin_loop() {}
#endif
```

```
loop {
    if seq_num > self.frame_idx {
        return frame;
    }

    std::hint::spin_loop();
}
```

```
while True:
    if seq_num > self.frame_idx:
        return frame
    pass
```

Listing 2: Wait-free spin-loop implementations. C++ (top) and Rust (middle) use micro-architectural hints to pause execution. Python (bottom) instead relies on an empty pass loop that holds the GIL.

5.5. Channels

Channels were required to enable communication between the inference and late-fusion threads, and the late-fusion and telemetry threads. The former uses an MPSC channel, where multiple producers (the inference threads) push data into the channel to be pulled by one consumer (the late-fusion thread). The latter uses a Single-Producer Single-Consumer (SPSC) channel, where one producer (the late-fusion thread) pushes data into the channel, to be pulled by one consumer (a telemetry thread).

Rust provides a memory-safe bounded channel via `std::sync::mpsc`. However, C++ required a third-party library (`moodycamel::BlockingConcurrentQueue`). Because this implementation is unbounded and dynamically allocates memory to grow, a `std::counting_semaphore` was utilised to enforce a maximum capacity, satisfying the zero-allocation pipeline requirement.

The Python channel implementation utilised the standard library’s `queue.Queue`. This class uses mutexes to lock the queue, preventing race conditions each time an attempt is made to push or pull from the channel. This constant locking and unlocking introduces context-switching (which in turn introduces latency and jitter), and thrashes the GIL, degrading performance of other threads as they are starved of execution time.

5.6. Bounded Queue Locking

The bounded buffer queue is shared between the bridge and inference threads, where the former pushes data into the queue, and the latter pops data from it. This required the use of a mutex lock to enforce memory safety and to guarantee mutual exclusion. In C++ and Python, the locks were implemented as member variables of the Queue class (`std::mutex` and `threading.Lock`, respectively). While C++ uses Resource Acquisition Is Initialisation (RAII), and Python

uses a context manager to set up and tear down the mutex automatically, neither offers enforced linking of the lock to the queue. Conversely, Rust's `std::sync::Arc<Mutex<T>>` combines compiler-enforced RAII and data ownership, guaranteeing that the queue cannot be accessed without first acquiring the lock guard.

5.7. Zero-Allocation Telemetry

To prevent the telemetry thread from becoming a confounder of the results that are being measured (i.e. the “observer effect”), it was designed to be zero-allocation and non-blocking.

Triple-buffering was implemented to exchange the telemetry epoch from the hot thread to the telemetry thread. As demonstrated in Listing 3, a pointer to the active epoch is swapped with a pointer to a clean epoch, with no I/O latency or dynamic memory allocation.

```
void
TelemetryWriter::swap_buffers()
{
    last_swap_ = std::chrono::steady_clock::now();
    auto next_epoch_opt = receiver_.try_receive();

    if (next_epoch_opt.has_value()) {
        sender_.send(std::move(current_epoch_));
        current_epoch_ = std::move(next_epoch_opt.value());
    }
}
```

```
fn swap_buffers(&mut self) -> Result<> {
    self.last_swap = Instant::now();

    if let Ok(mut epoch) = self.receiver.try_rcv() {
        std::mem::swap(&mut self.current_epoch,
            &mut epoch);
        self.sender.send(epoch)?;
    }

    Ok(())
}
```

```
def swap_buffers(self):
    self.last_swap = time.perf_counter_ns()
    epoch = self.receiver.try_receive()
    if epoch != None:
        self.sender.send(self.current_epoch)
        self.current_epoch = epoch
```

Listing 3: The C++ (top), Rust (middle), and Python (bottom) implementations of the zero-allocation telemetry thread buffer swap. The hot thread pushes the current epoch to the telemetry thread, and swaps in a clean epoch for the next telemetry cycle.

5.8. Memory Telemetry

To measure Python’s GC jitter, the GC callback hook was utilised, as shown in Listing 4. This allows the duration of “stop-the-world” events to be measured without the overhead of `tracemalloc` or other monitoring tools.

```
def gc_callback(phase, info):
    global pause_ns

    if phase == "start":
        _state["start_time"] = time.perf_counter_ns()
    elif phase == "stop":
        pause_ns += (time.perf_counter_ns() -
            _state["start_time"])

gc.callbacks.append(gc_callback)
```

Listing 4: Capturing the duration spent in the Python GC using the `gc.callbacks` hook.

To measure memory efficiency without the overhead of third-party tools, the C++ and Rust memory allocators were overridden (Listing 5), and relaxed memory ordering was used to prevent unnecessary stalls of the pipeline.

```
void*
operator new(std::size_t count)
{
    if (telemetry::track_allocations) {
        telemetry::allocated_bytes.fetch_add(count,
            std::memory_order_relaxed);
        telemetry::allocation_count.fetch_add(1,
            std::memory_order_relaxed);
    }

    if (void* ptr = std::malloc(count)) {
        return ptr;
    }

    throw std::bad_alloc{};
}

void
operator delete(void* ptr, std::size_t count) noexcept
{
    if (telemetry::track_allocations) {
        telemetry::freed_bytes.fetch_add(count,
            std::memory_order_relaxed);
    }

    std::free(ptr);
}
```

```
unsafe impl GlobalAlloc for TrackingAllocator {

    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let track = TRACK_ALLOCATIONS.try_with(|t|
            t.get()).unwrap_or(true);

        if track {
            ALLOCATED_BYTES.fetch_add(layout.size(),
                Ordering::Relaxed);
            ALLOCATION_COUNT.fetch_add(1,
                Ordering::Relaxed);
        }

        unsafe { System.alloc(layout) }
    }

    unsafe fn dealloc(&self, ptr: *mut u8,
        layout: Layout) {
        let track = TRACK_ALLOCATIONS.try_with(|t|
            t.get()).unwrap_or(true);

        if track {
            FREED_BYTES.fetch_add(layout.size(),
                Ordering::Relaxed);
        }

        unsafe { System.dealloc(ptr, layout) }
    }
}
```

```
#[global_allocator]
static GLOBAL: TrackingAllocator = TrackingAllocator;
```

Listing 5: Overriding the global memory allocation and deallocation functions in C++ (top) and Rust (bottom) to capture memory allocation metrics.

6. Results

To prevent cold-start initialisation from skewing the steady-state measurements, the first 10 seconds of all telemetry logs were excluded from the diagrams and analyses unless otherwise stated. Similarly, unless otherwise stated, all evaluations were performed in MAXN_SUPER power mode (using `nvpmode1 -m 2`).

6.1. Baseline Performance (MAXN_SUPER)

With MAXN_SUPER mode enabled (Figure 12), both C++ and Rust were able to sustain ingestion without absolute saturation up to a load multiplier of 4.0 for the Bounded Queue policy, and 5.5 for Exponential Backoff. For the static load-shedding policies (Drop Oldest and Drop Newest), C++ was able to process the data streams at a load multiplier of 1.0, while Rust successfully processed them at 1.5. When using Adaptive Decimation, both compiled languages reached terminal saturation at 1.0.

Conversely, Python was only able to process the data streams at 4% (load multiplier of 0.04) when Exponential Backoff was used. Absolute saturation was reached at < 0.01 for all other backpressure policies.

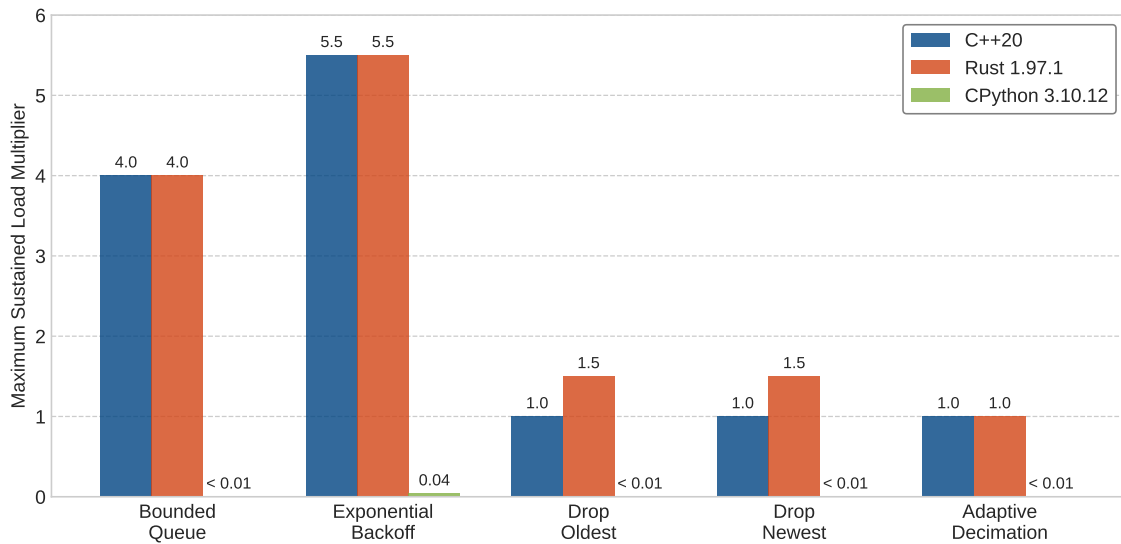


Figure 12: Absolute pipeline saturation points for each language and backpressure policy.

A Cumulative Distribution Function (CDF) graph was used to analyse the 99.9th percentile latency of the RGB anchor stream when using the Exponential Backoff backpressure policy with a load multiplier of 5.5 (Figure 13). This represents the maximum measured throughput sustained by both compiled languages without breaching the 100 ms latency deadline. Both compiled languages had similar latency distributions. However, despite using the same flow-control policy, Python consistently breached the deadline with a maximum latency of 954.7 ms.

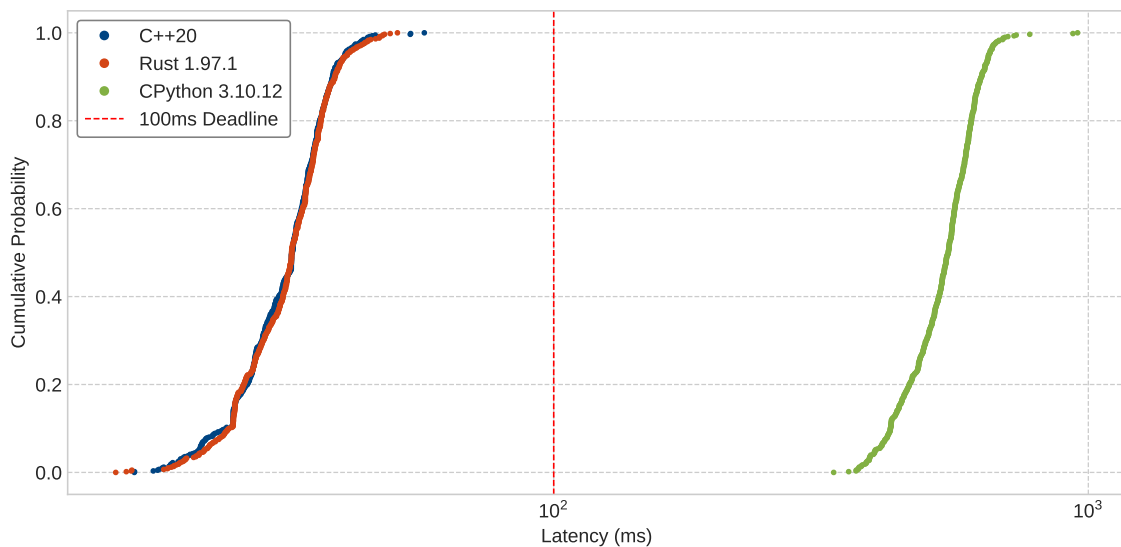


Figure 13: CDF of 99.9th percentile total latency for each language at load 5.5, using the Exponential Backoff backpressure policy.

The 99.9th percentile latency distribution was also analysed using the Exponential Backoff backpressure policy at a load multiplier of 7.0 (Figure 14), which represents the lowest measured throughput at which the compiled languages breached the 100 ms latency deadline. Python was omitted from this comparison to preserve visual clarity.

While the flow-control policy does prevent data loss (until the 1-second unbound circular buffer laps), it does so at the cost of increased queue accumulation and latency exceeding the 100 ms deadline. Specifically, 90.6% of the C++ epochs breached the deadline with a maximum latency of 177.7 ms, compared to 92.3% of the Rust epochs with a maximum latency of 174.7 ms.

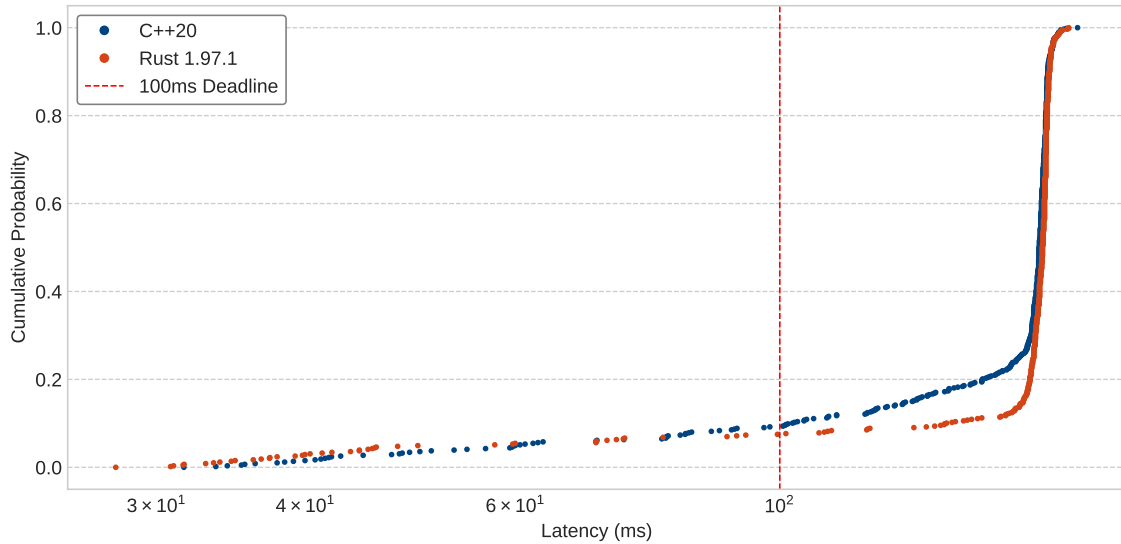


Figure 14: CDF of 99.9th percentile total latency for both compiled languages at Load 7.0, using the Exponential Backoff backpressure policy.

6.2. Individual Stream Saturation

Asymmetrical stream saturation was observed when the Exponential Backoff policy was used, under which both compiled implementations achieved a maximum load of 5.5 (Section 6.1).

As demonstrated in Figure 15, this throughput limitation is isolated to the RGB stream. While the pipelines began to drop frames from the 30 Hz RGB stream when the load exceeded 5.5, both the 1.6 kHz Accelerometer stream and 2.0 kHz Gyroscope stream successfully sustained ingestion without data loss up to the maximum tested load multiplier of 20.0.

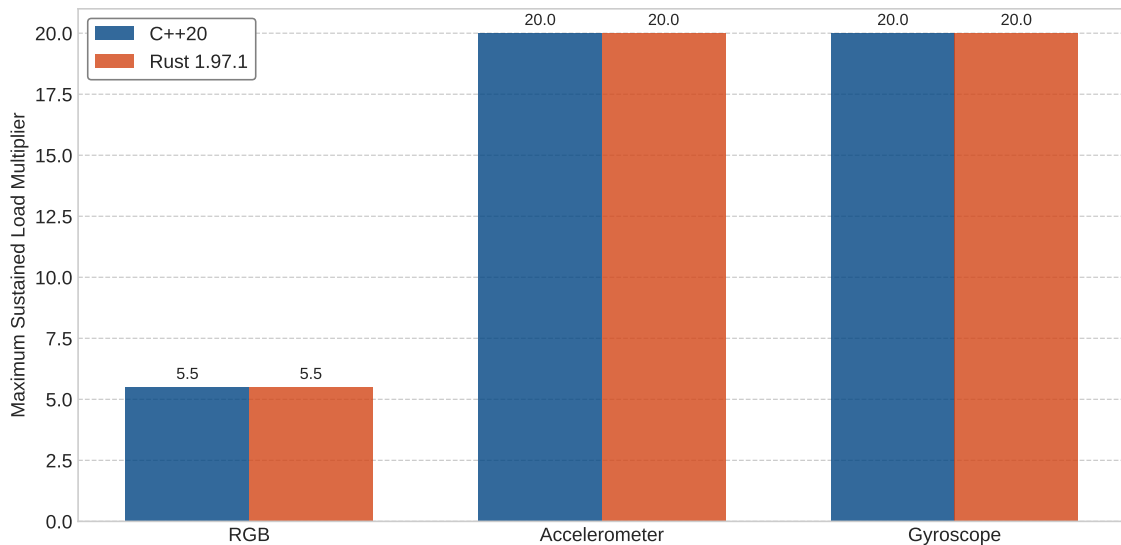


Figure 15: Maximum load multiplier sustained by each individual stream without dropping frames, using the Exponential Backoff backpressure policy.

6.3. Latency Breakdown

The overall pipeline latency was broken down into its five stages, and the average median (p50) was calculated. Summing the deep tail percentiles (e.g. the 99.9th) across all stages would be misleading, as it would assume that the worst-case latency of every stage occurs on the exact same frame, which would misrepresent the actual latency of the typical frame. The Exponential Backoff policy was selected, with a load of 0.04, as this was the maximum throughput that all three implementations were able to sustain without dropping frames.

As Figure 16 shows, C++ and Rust both completed the pipeline well within the 100 ms deadline (7.6 ms and 7.3 ms respectively), with the inference execution stage taking the majority of that time (5.7 ms for C++, and 5.6 ms for Rust). Conversely, Python's total average median latency breached the 100 ms deadline (105.1 ms), with a significant portion of that time spent with frames waiting in the unbounded circular buffer (24.3 ms) or the bounded queue buffer (24.4 ms). The inference execution stage (24.3 ms) was also significantly slower than in the compiled languages.

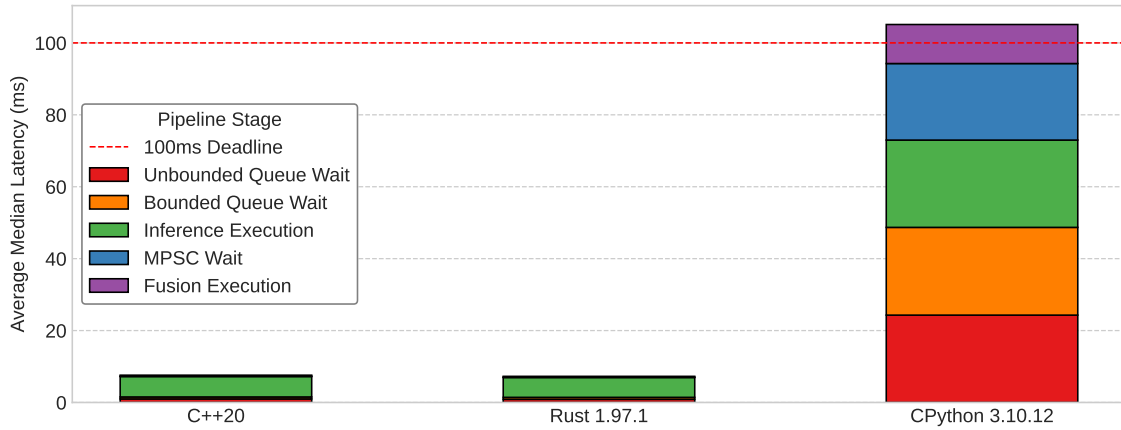


Figure 16: Stage-by-stage median (p50) latency breakdown for the RGB stream at load 0.04 using the Exponential Backoff backpressure policy.

6.4. Flow Control vs. Load Shedding

To compare the total number of dropped or lapped frames between flow-control and load-shedding policies, the most efficient flow-control policy (Exponential Backoff) was contrasted against the two static load-shedding policies (Drop Oldest and Drop Newest). These policies both drop frames when the bounded queue is full, without trying to prevent it from filling to capacity by dynamically adjusting the flow-rate. Rust was the most efficient implementation overall, and so was selected for this comparison. A load multiplier of 2.5 was used as it is the lowest measured multiplier at which both load-shedding policies dropped frames and showed significant divergence during the evaluation period.

As shown in Figure 17, the Exponential Backoff flow-control policy was able to fully absorb the latency jitter by taking advantage of the 1-second unbounded buffer. In contrast, the load-shedding policies dropped frames to maintain the 100 ms latency deadline. The Drop Oldest policy recorded a total of 11 dropped frames over the 600-second evaluation period, while the Drop Newest policy recorded 57 dropped frames.

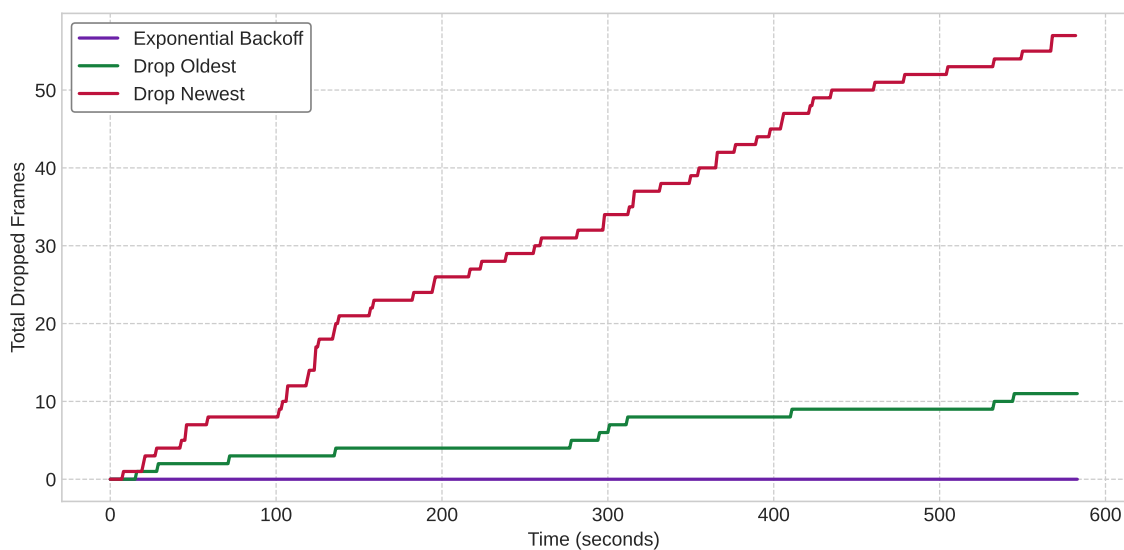


Figure 17: Total dropped frames for Rust at Load 2.5, comparing the Exponential Backoff flow-control policy against the Drop Oldest and Drop Newest load-shedding policies.

While flow-control policies excel at preventing data loss when experiencing jitter, load-shedding policies sacrifice the data to meet the latency deadline. As shown in Figure 18, the impact of retaining stale data to prevent loss forced the majority of epochs (92.3%) to breach the 100 ms latency deadline (up to 174.7 ms) when the pipeline reached saturation at a load multiplier of 7.0. Conversely, by dropping frames and preventing a growing backlog, the load-shedding policies guarantee that surviving frames are processed within the deadline.

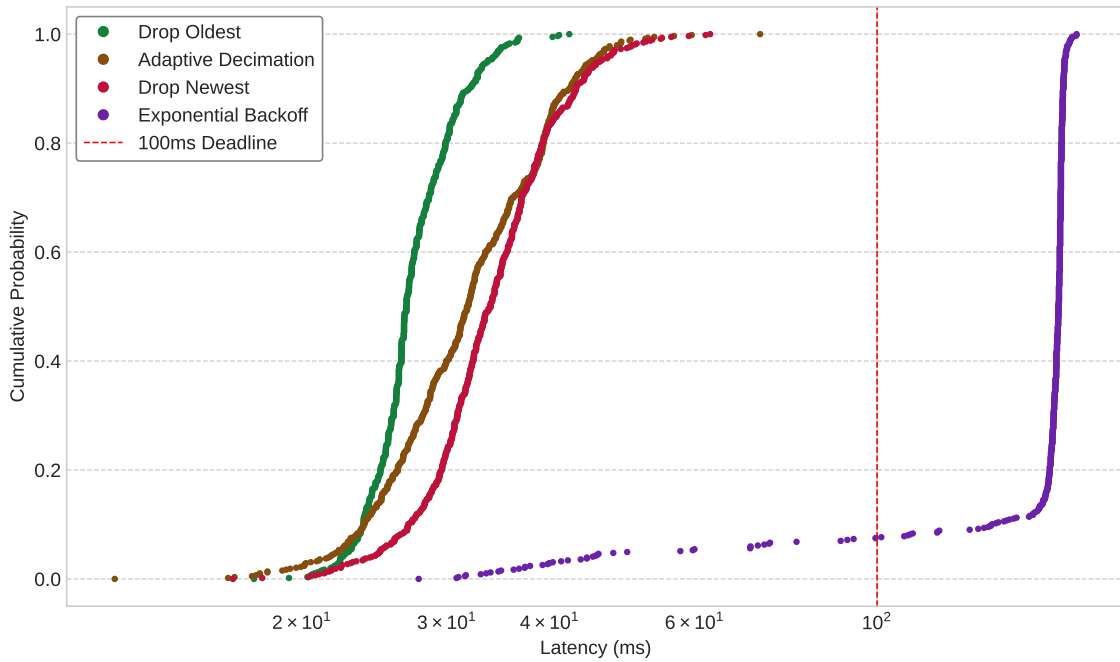


Figure 18: Latency comparison for Rust at load 7.0, comparing the Exponential Backoff flow-control policy against the Drop Oldest, Drop Newest, and Adaptive Decimation.

To mitigate the temporal data loss characteristic of load-shedding policies, while also avoiding the fast growth of stale frames within the unbounded buffer, the Adaptive Decimation policy was compared. By dynamically downsampling the incoming stream before the bounded buffer reaches full capacity, the policy reduces volume pressure while maintaining temporal continuity. However, preserving the temporal continuity sacrifices a substantial number of frames — 522 frames were dropped over the 600-second evaluation period (not shown in Figure 17 to preserve visual clarity). In addition, total latency increased compared to the Drop Oldest policy, though it was lower than that achieved by Drop Newest. Because Adaptive Decimation and Drop Newest both drop incoming frames, the existing frames in the bounded buffer continue to age while waiting to be processed. Conversely, Drop Oldest drops the oldest data from the bounded buffer, guaranteeing that the freshest data survives.

6.5. Memory Overhead

To investigate the impact of Python’s automated memory management on deadline adherence, the maximum latency was plotted for all three implementations, alongside the recorded duration of Python’s Garbage Collection (GC) pauses. A load multiplier of 0.04 was selected, with a policy of Exponential Backoff, as this was the maximum throughput measured that all three implementations were able to sustain without the loss of data. The initial 60-second window, as illustrated in Figure 19, captures the initialisation phase and the subsequent steady-state, while retaining visual clarity.

C++ and Rust demonstrated stable maximum latencies below the 100 ms deadline during the initialisation phase (27.3 ms and 30.8 ms, respectively) and the subsequent steady-state phase (40.5 ms and 40.0 ms), with ranges of 35.7 ms and 33.9 ms, respectively, during steady-state.

During the initialisation phase, Python exhibited a maximum latency of 1535.1 ms. During the subsequent steady-state phase, the maximum latency increased to 2210.4 ms, with a range of 2164.6 ms. Python’s “stop-the-world” GC events were confined to the first few seconds of the 60-second window. This was confirmed by a Spearman’s rank correlation (ρ) across the steady-state window, which produced an undefined result (NaN) due to no GC events occurring during that time.

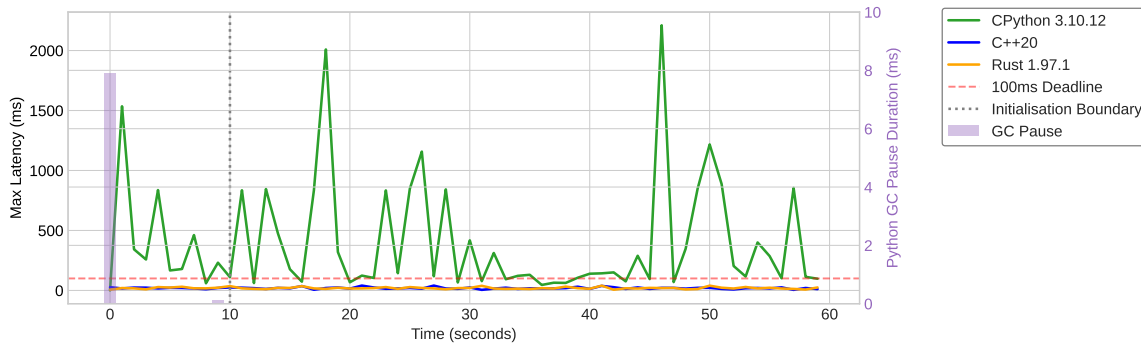


Figure 19: Maximum latency vs. GC pause duration over the first 60 seconds of execution.

To further evaluate the resource efficiency of the runtime models, the RSS was measured under the heaviest load conditions that both compiled languages were able to sustain without dropping frames (Exponential Backoff, load 5.5).

As demonstrated in the left panel of Figure 20, the overall memory footprint remained consistent for C++ and Rust, which stabilised at approximately 748.3 MiB and 745.6 MiB, respectively. Conversely, Python’s memory footprint had a step-increase at approximately 250 seconds, stabilising at 793.8 MiB. Because a load of 5.5 greatly exceeds Python’s maximum sustainable throughput of 0.04, the step-up suggests internal memory fragmentation.

The right panel of Figure 20 demonstrates the cumulative dynamic memory allocations of the C++ and Rust implementations in the pipeline. Following the 10-second initialisation phase, there was no dynamic memory allocation for either implementation. This was confirmed by a Spearman’s rank correlation (ρ) comparing dynamic memory allocation and CPU temperature across the steady-state window, which produced an undefined result (NaN) for both implementations due to no dynamic memory allocation occurring during that time.

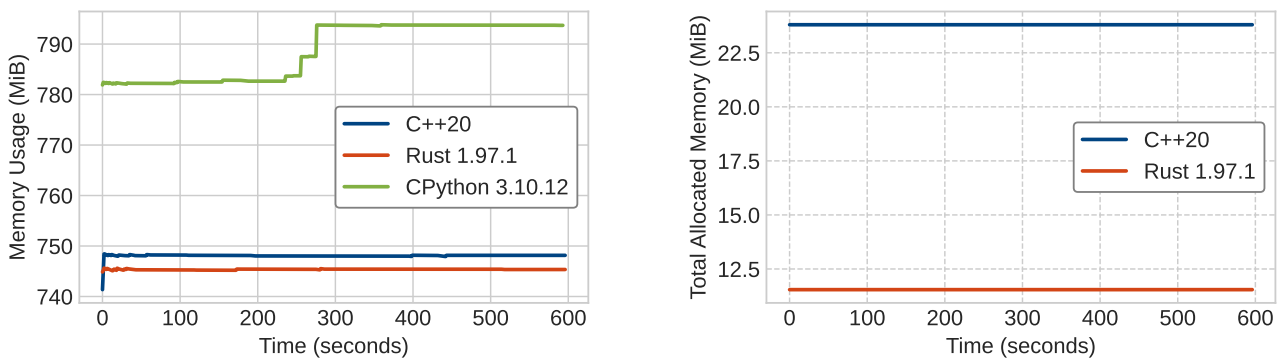


Figure 20: Memory profiling during steady-state execution. The left panel compares the RSS footprint. The right panel shows the total dynamic memory allocations by the C++ and Rust implementations.

Heap fragmentation using the `fordblks` field from `mallinfo2()` was also measured to evaluate the long-term stability of the runtime models and the effectiveness of the zero-allocation architecture. The recorded telemetry showed that heap fragmentation during the steady state was negligible for both C++ (5.0 KiB) and Rust (10.6 KiB). Coupled with RSS footprints of under 750 MiB, and steady-state dynamic allocation rates of 0.0 Bytes/second, this proves that both implementations were able to sustain the load without memory churn or significant fragmentation.

6.6. Impact of Hardware Power Constraints (7W Mode)

A second evaluation was executed using the Jetson Orin Nano's 7-Watt power profile (using `nvpmodel -m 3`) which disables two of the six CPU cores, caps the CPU frequency at 960 MHz, and restricts the GPU clock to a maximum of 408 MHz (see Appendix B). This is in comparison to the unconstrained limits of the MAXN_SUPER power mode.

As shown in Figure 21, utilising the 7-Watt power mode reduced the maximum throughput for the compiled implementations when using flow-control policies. For both languages, the maximum sustained load multiplier for Bounded Queue fell from 4.0 to 2.0, and for Exponential Backoff, it fell from 5.5 to 2.5. The static load-shedding policies were unaffected, with C++ and Rust sustaining load multipliers of 1.0 and 1.5 respectively, identical to their unconstrained MAXN_SUPER performance. Similarly, both compiled languages were able to sustain a load multiplier of 1.0 for Adaptive Decimation — also identical to their unconstrained performance.

The Python implementation saw a small improvement in maximum throughput across all backpressure policies. Bounded Queue increased from < 0.01 to 0.02, Exponential Backoff increased from 0.04 to 0.06, and the remaining policies increased from < 0.01 to 0.01.

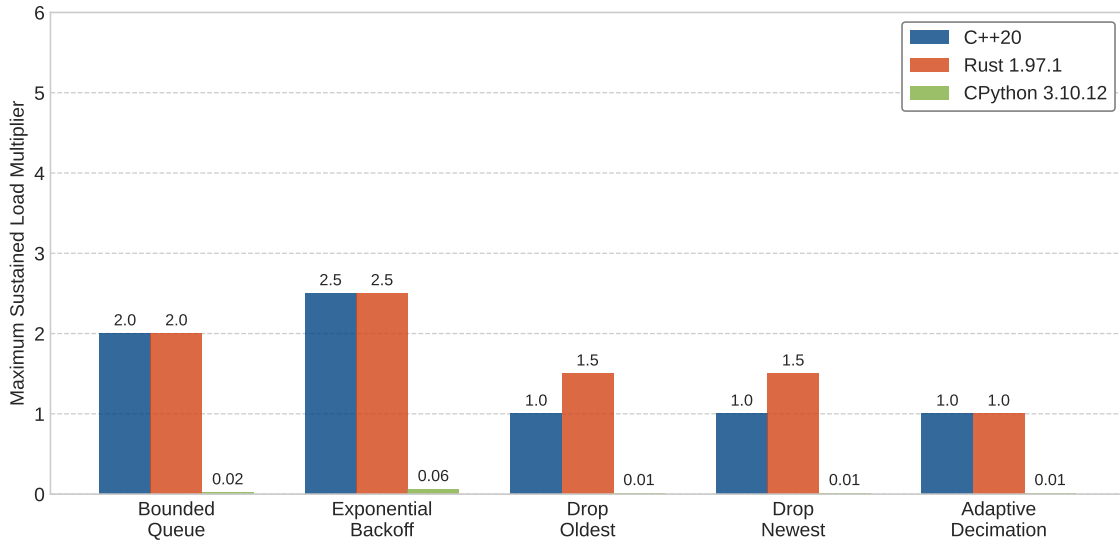


Figure 21: Absolute pipeline saturation points of each language and backpressure policy under the 7-Watt power constraint.

Latency was also impacted when utilising the 7-Watt power profile. Using an Exponential Backoff policy with a load multiplier of 5.5 was the maximum ingestion rate that both C++ and Rust were able to sustain in MAXN_SUPER mode without dropping frames. However, as demonstrated in Figure 22, when the power was constrained to 7 Watts, the 99.9th percentile latency times of both implementations increased from maximums of 57.3 ms and 51.0 ms to 285.2 ms and 300.9 ms respectively, with every epoch breaching the 100 ms deadline.

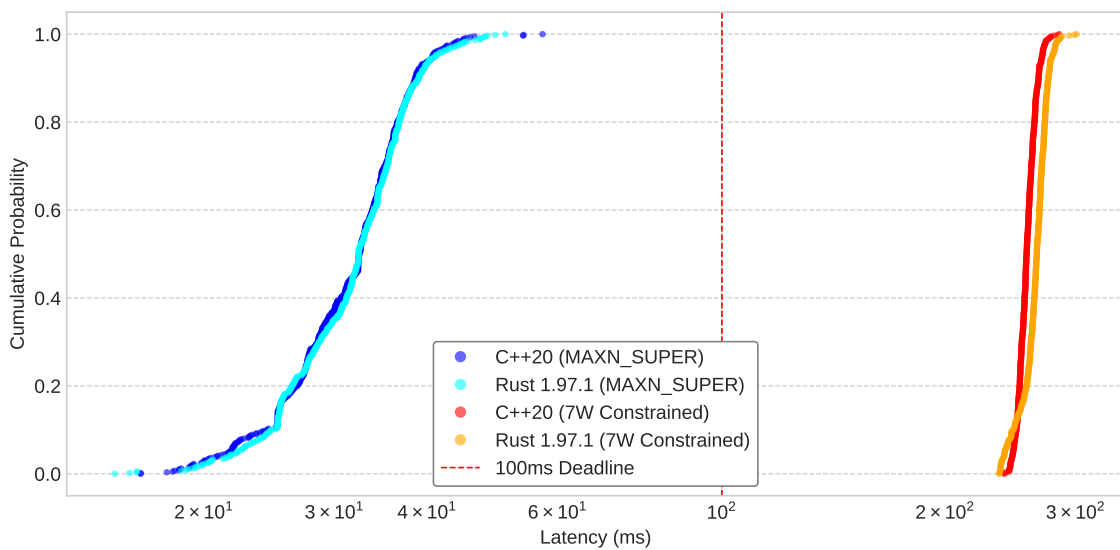


Figure 22: CDF of 99.9th percentile total latency for both compiled languages at Load 5.5, using the Exponential Backoff backpressure policy comparing MAXN_SUPER (unconstrained) and 7-Watt (constrained) power profiles.

6.7. Thermal Accumulation

During the evaluations, the Jetson Orin Nano's emergency fan cooling prevented DVFS frequency scaling by engaging the fan at 74°C. However, analysis of the `tegrastats` telemetry revealed thermal accumulation differences between the runtime models.

Figure 23 shows all three implementations subject to the native stream rate (load 1.0) using the Exponential Backoff backpressure policy. Because this is only a fraction of the maximum capacity for C++ and Rust, both compiled languages processed the data efficiently and spent the majority of the epoch yielding (utilising micro-architectural pause instructions). Consequently, neither triggered the cooling fan, with C++ and Rust stabilising at approximately 64.0°C and 60.5°C respectively. Conversely, Python triggered the cooling fan at approximately 143 seconds, allowing the temperature to stabilise at approximately 57.0°C.

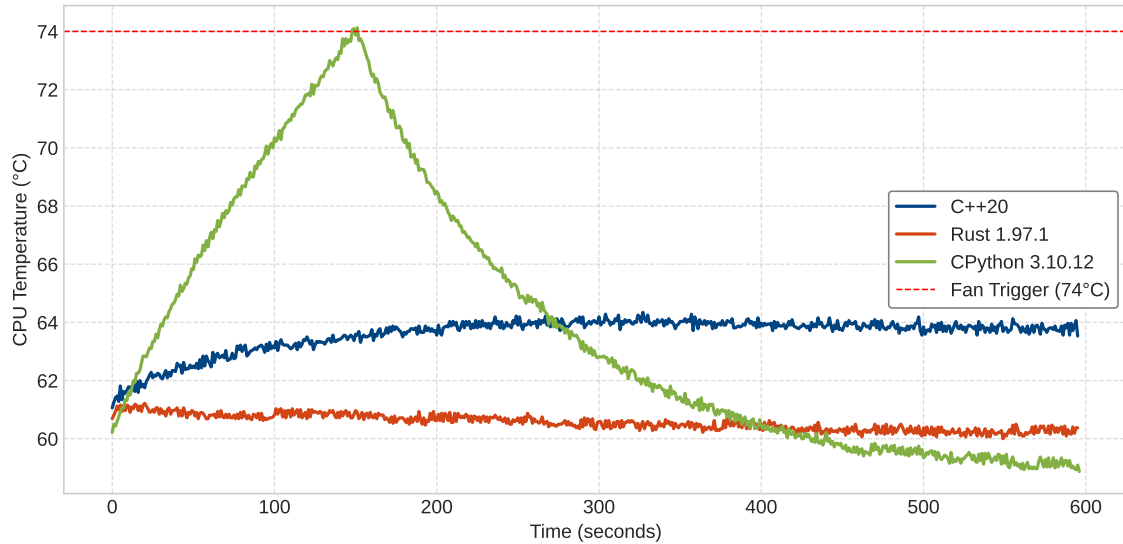


Figure 23: Thermal accumulation at the native stream rate using the Exponential Backoff backpressure policy.

Figure 24 plots the temperature curves of each implementation at their respective maximum measured saturation points (Exponential Backoff, load 5.5 for the compiled languages, and 0.04 for Python). At maximum throughput, C++ and Rust triggered the fan after 63 seconds and 72 seconds respectively. The temperatures then settled to approximately 72.0°C and 69.5°C respectively. Conversely, at the maximum sustainable load multiplier of 0.04, the fan triggered for Python after 104 seconds, and the temperature then settled to approximately 57.0°C.

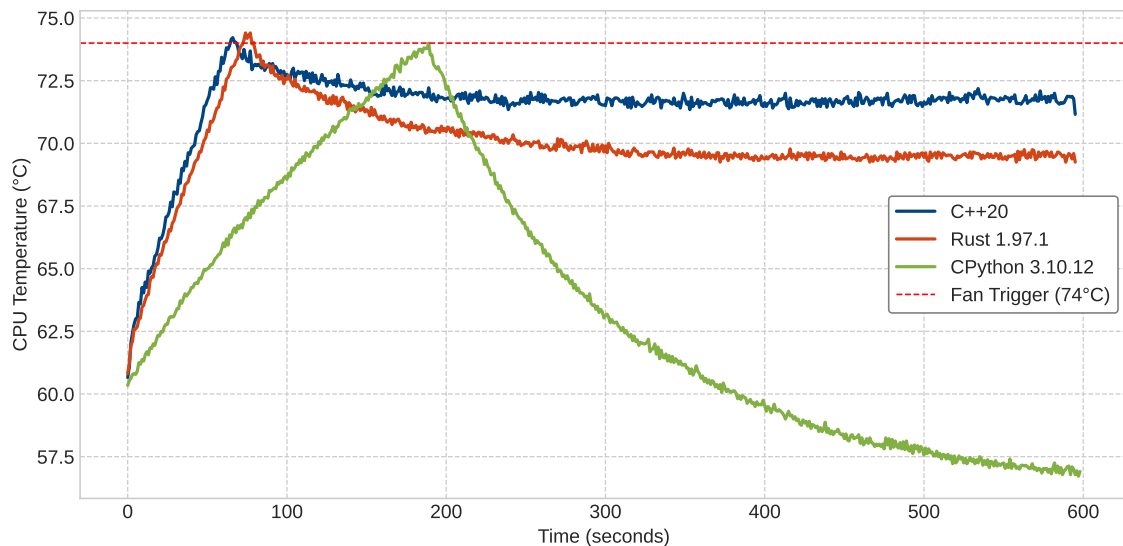


Figure 24: Thermal accumulation at the maximum sustainable throughput using the Exponential Backoff backpressure policy.

6.8. Statistical Analysis of Latency

To evaluate how much latency can be attributed to the runtime models, a Kruskal-Wallis H-test was performed on the steady-state median (p50) latency measurements. This determines how likely it is that latency differences were caused by the choice of language, and not operating system interrupts. A load multiplier of 0.04 using the Exponential Backoff policy was used, as this was the maximum ingestion rate that all three implementations were able to sustain without data loss.

The test revealed a significant difference in performance between the languages ($H = 696.64$, $p < 0.001$). Furthermore, the effect size ($\epsilon^2 = 0.6660$) indicated that approximately 66.6% of the variance in processing speed was caused by the runtime model itself, and not by random system noise.

A Dunn's post-hoc pairwise comparison with a Bonferroni correction (Table 1) was performed to identify which languages differed. The results showed no significant difference between C++ and Rust ($p = 1.000$), proving that baseline performance between these two languages is statistically identical. Conversely, Python was significantly slower than both C++ and Rust ($p < 0.001$), confirming that the choice of runtime model has a significant impact on the overall latency of the pipeline.

Runtime Model	C++	Rust	Python
C++	—	$p = 1.000$	$p < 0.001$
Rust	$p = 1.000$	—	$p < 0.001$
Python	$p < 0.001$	$p < 0.001$	—

Table 1: Dunn's post-hoc pairwise comparison. The Kruskal-Wallis test indicated a statistically significant difference in median latency between runtime models.

6.9. Impact of Hardware Power Constraints (7W Mode)

Severe latency degradation was recorded when the Jetson Orin Nano's power mode was restricted to 7 Watts (using `nvpmodel -m 3`). A Spearman's rank correlation was performed on both compiled implementations to determine if increasing CPU temperature also increased latency. Because Python did not experience latency degradation under the 7-Watt profile, it was excluded from this correlation. A load multiplier of 5.5 was used with the Exponential Backoff policy, as this was the maximum ingestion rate that both compiled languages were able to sustain without dropping frames.

The calculated results showed no significant correlation for either C++ or Rust. ρ was very close to zero for both languages (-0.0134 and 0.0382 respectively), indicating no relationship between CPU temperature and latency. This was further confirmed by the high p-values (p) of 0.746 and 0.356 respectively. Because the Jetson's fail-safe thermal management forcefully triggered the cooling fan at 74°C, DVFS throttling was prevented. Therefore, the results confirm that the performance degradation was a result of static resource constraints (i.e. a reduced number of CPU cores and lower clock frequencies), and not DVFS throttling caused by thermal accumulation.

7. Discussion

7.1. Compilation Times

Despite Rust's stricter compile-time checks and safety guarantees (e.g. ownership, the borrow checker, and strict variable usage), compilation times were significantly faster than experienced with C++. The latter implementation relies on several template classes, both standard (e.g. `std::shared_ptr`, `std::vector`) and pipeline specific (the channel Sender/Receiver and the Queue). By language design, C++ templates must be defined in header files, and because C++ relies on a pre-processor source file inclusion model (`#include`), these headers are copied into every Translation Unit (TU) that references them, forcing the compiler to repeatedly parse the same templates across multiple TUs.

Conversely, Rust's compiler does not rely on source file inclusion, and instead parses each source file only once (regardless of how many times it is imported), preventing an accumulation of unnecessary parsing overhead. Furthermore, the cargo build tool caches a project dependency graph to avoid re-parsing or re-compiling unchanged files.

In contrast to C++ and Rust, Python is an interpreted language and consequently has no compilation overhead, thus reducing friction during initial prototyping. However, errors that the other languages would catch at compile time are only discovered at runtime in Python, risking reduced system stability within the production environment.

7.2. Error Handling

Rust's `Result` type forces developers to explicitly handle failure states at compile time. The `?` propagation operator offers a concise mechanism to push errors up the call stack without complicating the primary logic flow. These language paradigms simplify the error-handling code, and allow the adoption of boilerplate-reducing third-party crates (e.g. `anyhow`) to add error handling and contextual information in one expression.

In contrast, both the C++ and Python implementations rely on developer discipline to write verbose error handling code to explicitly handle exceptions (`try/catch` and `try/except`) and legacy error codes, risking the occurrence of unhandled errors (decreasing system stability), and a lack of contextual information when debugging (increasing system maintenance overhead). Achieving Rust's level of granularity in C++ or Python would require a `try`-block around every invocation of a function that may throw an exception. This would severely hamper code readability and maintainability, and therefore idiomatic C++ and Python implement coarser-grained exception handling, at the sacrifice of diagnostic information when an error occurs.

7.3. Language Ergonomics

While static analysis tools such as Lizard provide a quantitative evaluation for Non-commenting Lines Of Code (NLOC), and a Cyclomatic Complexity Number (CCN), they do not consider the qualitative experience of the developer when writing and maintaining code. The implementation of the three pipelines revealed significant differences in language ergonomics and the resulting cognitive load. Both Rust and Python provide simple mechanisms for transforming data collections and evaluating enumeration types (such as Rust's `match` operator), requiring minimal boilerplate code and improving code readability.

While C++ has evolved over successive versions to offer similar functionality via the standard library (e.g. `std::ranges` and `std::transform` for collections, or `std::variant` and `std::visit` for enumerations), utilising these features was counterproductive. For example, extracting the minimum frame rate from a collection of stream configurations in Rust was achieved with a simple, concise expression:

```
let min_fps = configs.iter().map(
    |(queue, _, _)| queue.fps).min().unwrap();
```

Achieving an equivalent transformation in C++ requires the use of C++20 `std::ranges` and lambda expressions. Because C++ lacks concise closure syntax, the mapping operation requires a full lambda expression nested within a view pipeline:

```
const auto min_fps = std::ranges::min(
    configs | std::views::transform([](const auto& cfg)
    {
        return cfg.queue.fps;
    })
);
```

Consequently, traditional `for` loops were used instead to maintain code readability, and to reduce maintenance overhead:

```
size_t min_fps = std::numeric_limits<size_t>::max();

for (const auto& cfg : configs) {
    min_fps = std::min(min_fps, cfg.queue.fps);
}
```

Similarly, pattern matching in C++ required the use of `std::visit`, which introduced syntactic complexity that made the code difficult to read. The code formatter struggled to parse the code coherently, requiring `// clang-format off` directives to disable the formatter and maintain legibility. This demonstrates that the complexity of C++'s legacy architecture and backward compatibility can deter the adoption of attempts to introduce modern approaches to software development, forcing developers to revert to a more traditional style of programming.

While features such as dynamic typing and automated memory management are often believed to reduce cognitive load, this evaluation revealed that they can introduce additional friction during runtime. For example, during the implementation of the Adaptive Decimation backpressure policy (Section 5.3), the strictly typed C++ and Rust runtime models truncate the result of the integer division. Conversely, Python's division operator (`/`) implicitly casts the result to a `float`, causing the subsequent modulo operation to silently fail, resulting in virtually all frames being dropped. Instead, to truncate the division result, the floor-division operator (`//`) was required. This demonstrates that while features such as dynamic typing may reduce developer friction and boilerplate code, they shift the burden of semantic validation to runtime, increasing debugging overhead in complex systems.

7.4. Ecosystem Maturity and Deployment

The C++ ecosystem is a highly cohesive, mature development environment. Its foundation libraries have remained stable for years, with a strong adherence to backwards compatibility offering long-term stability and reducing long-term maintenance overhead.

While Rust's cargo build tool offers easy-to-use package management, the maturity of the ecosystem is still evolving. In contrast to C++, many of Rust's foundation libraries (e.g. `libc`, `nix`) remain in "zerover" (0.x.y) pre-release states. This results in a cognitive burden caused by API changes, deprecated or incomplete documentation, and conflicting online resources.

Furthermore, whereas C++ uses system-installed shared libraries, meaning they are *layer cached* early in the Docker image build process, Rust's cargo downloads and compiles all dependencies during the build process, resulting in extended deployment times. Circumventing this required dummy source files to be created (with an identical `Cargo.toml` manifest) to allow a layer to be cached early, thus mitigating download-and-compile overhead during every

deployment. It was also necessary to artificially update the modification timestamps of the real source files to guarantee that the later `cargo build` command would recompile the source files, rather than using the cached dummy executables.

Python's large ecosystem accelerates initial prototyping, but introduces friction during deployment. While C++ and Rust compile down to stand-alone executable files which can be easily deployed to the target hardware, Python's interpreted nature demands deployment of the Python interpreter itself, the source files, and an identical virtual environment. This results in Python deployment being heavier, more complex, and more fragile than the statically compiled counterparts. Furthermore, Python's package management system was found to be fragile. Package versions had to be manually pinned (by referring to the Python Package Index (PyPI) release history [43]) to prevent versions being automatically selected that were incompatible with the Python interpreter under evaluation (CPython 3.10.12). In addition, the ONNX Runtime package for the Jetson Orin Nano is remotely hosted on NVIDIA's PyPi server [44], which was often found to be unavailable, requiring the package to be manually downloaded and saved locally for adding to the Docker image later.

It should be noted that all pipeline implementations were containerised to ensure a consistent environment for the evaluation suite, and so the benefits of a stand-alone executable were not realised. However, in a production Edge-AI environment where containerisation is not desirable, the complexity and fragility of deployment should be a major consideration.

7.5. Concurrency and Memory Safety

A contrast was experienced between the manual memory management of C++ and the compiler-enforced memory safety of Rust. Spawning threads using `std::jthread` in C++ relies on lambda expressions, which allows variables to be captured by reference (e.g. `[&receiver]`). While concise and convenient for developers, this creates a dangling reference if the spawned thread uses the captured variable after the variable's enclosing scope has ended. Because the C++ compiler does not check for memory safety issues, this error is not flagged to the developer. Therefore, avoiding such errors relies on developer discipline and vigilance, which becomes an increasingly difficult burden as the size and complexity of the codebase grows.

Conversely, Rust's ownership model and borrow checker guarantees memory safety. In the aforementioned example, the Rust compiler would refuse to allow references that may not outlive the spawned thread. Instead, Rust forces the developer to transfer ownership using the `move` keyword and atomic reference counting (e.g. `Arc<Mutex<T>>`). Though Rust's ownership model may be a steep learning curve for developers new to the language (similar to that experienced when transitioning from a functional paradigm to an object-oriented one), it eliminates memory-safety bugs that are notoriously difficult to resolve, shifting the burden from developer discipline and vigilance to compiler analysis.

Python's GC automatically handles the lifetime of the variable, preventing the dangling reference vulnerabilities of C++, without the steep learning curve of Rust's borrowing mechanism. However, this removes the developer's ability to determine when resources are reclaimed.

7.6. Memory Management Friction

It is generally believed that a runtime model's GC reduces cognitive burden when compared to manual memory management (C++), or an ownership model (Rust). However, when developing the temporal buffering of the unbounded queue's frame payloads, C++ provided the least friction. Because the developer is responsible for all memory allocation and deallocation, a frame's payload address can be stored directly in the frame itself, with no friction or risk of premature unmapping by the runtime model.

Rust's ownership model also prevents premature unmapping, but explicit unsafe blocks were required to satisfy the compiler when handling raw pointers. To allow memory addresses to be shared across threads, unsafe marker traits (`Send` and `Sync`) were necessary. Elsewhere, `Box<T>` was required to allocate variables on the heap (equivalent to C++'s `new` operator), and `std::cell::Cell<T>` was required to mutate a variable when the compiler's borrowing rules prevented exclusive access (functionally similar to C++'s `mutable` keyword). While these requirements help to highlight potential memory-safety issues, they also increase cognitive overhead compared to the almost frictionless, albeit less safe, raw pointer manipulation of C++.

Conversely, while Python's GC is designed to automate memory management, it increased friction when interfacing with shared memory. To circumvent the premature unmapping of shared memory buffer addresses when the bridge thread terminates and the inference thread is still processing frames, a memory location offset had to be provided in the Python frames instead of physical addresses. Furthermore, the inference threads had to maintain a second `multiprocessing.shared_memory.SharedMemory` object to read the frame payloads. Doing so caused a further issue of `KeyError` tracebacks displayed during evaluation, requiring `noop` monkey patching of the `resource_tracker.register` and `unregister` methods.

7.7. Asymmetrical Stream Saturation

Analysis of the individual data streams of the compiled implementations revealed a significant difference in the maximum sustainable ingestion speed when the Exponential Backoff policy was in use, with the RGB stream failing at a load multiplier of 7.0, and the Accelerometer and Gyroscope streams not dropping or lapping any frames at the maximum measured load multiplier of 20.0.

Little's Law ($L = \lambda W$) was used to enforce the 100 ms end-to-end latency deadline. Because the RGB stream's native speed is low at 30 Hz, its bounded buffer capacity is only 3 frames. In contrast, the 1.6 kHz Accelerometer stream buffer has a capacity of 160 frames, and the 2.0 kHz Gyroscope stream buffer has a capacity of 200 frames.

This asymmetry is exacerbated by both the inference thread and the late-fusion thread — the latter of which uses an MPSC channel anchored to the RGB stream. While the Accelerometer and Gyroscope inferences only occur every 53 and 66 frames respectively, inference is triggered for every RGB frame, which in turn triggers late-fusion, substantially increasing the average workload per frame for the RGB stream.

When the pipeline processes a high enough ingestion rate, the late-fusion thread will struggle to process the inference results from the MPSC channel. This causes the inference threads to block while attempting to push their results into the saturated channel. Because the IMU threads only push to the MPSC channel once per temporal window, they can continue to pop incoming frames from their bounded queues, accumulating the next window in local memory and giving the MPSC channel an opportunity to clear. This 1:53 and 1:66 insertion ratio gives the IMU bounded queues the elasticity to continue draining and absorb the temporary blockage without dropping frames. Conversely, because the RGB thread pushes to the MPSC channel on a 1:1 ratio, it blocks immediately. Unable to pop from its shallow 3-frame bounded queue, the RGB stream saturates almost immediately (see Figure 25). This triggers the active backpressure policy, resulting in dropped or lapped frames, and/or causing the latency deadline to be breached.

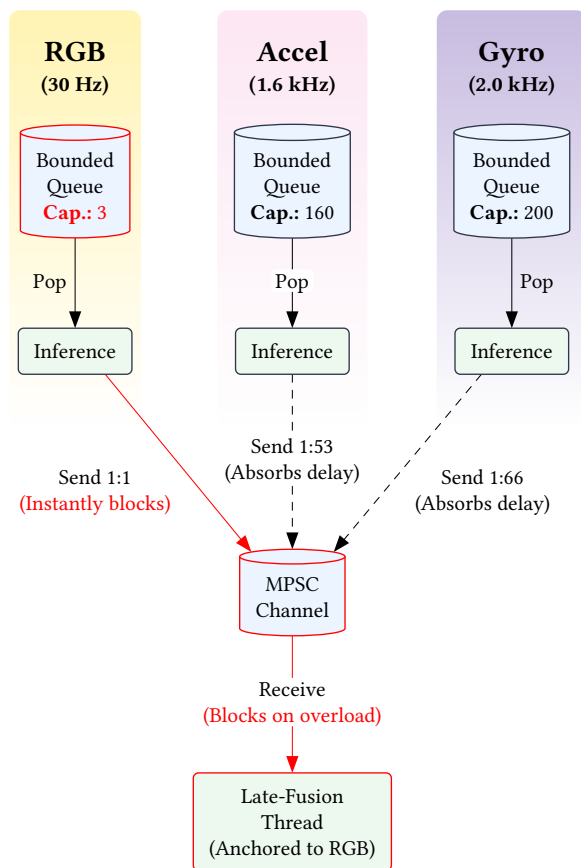


Figure 25: Visualisation of the MPSC late-fusion anchor bottleneck. The RGB stream's 3-frame capacity offers virtually no buffer elasticity, causing immediate upstream saturation when the MPSC channel blocks under heavy late-fusion load.

Because each backpressure policy has different mechanics, this asymmetry only occurs when the Exponential Backoff policy is used. All three of the load-shedding policies drop frames as soon as the bounded buffer is full, without utilising the elasticity of the 1-second temporal window of the unbounded buffer. Therefore, the elasticity of the inference temporal window does not affect the maximum sustainable throughput of the pipeline. Conversely, the Bounded Queue policy does utilise the unbounded buffer's elasticity, but because it constantly attempts to push frames into the bounded queue in a tight spin-loop, it causes contention on the queue's mutex lock, preventing the inference threads from pulling frames which would create the space that the bridge thread is waiting for.

7.8. Flow Control vs. Load Shedding

When choosing a backpressure policy, the data collected in this evaluation reveals a trade-off between temporal continuity and data preservation. Analysis of the number of dropped or lapped frames showed that the flow-control policies (Bounded Queue, Exponential Backoff) successfully preserved data frames at far higher ingestion rates than the policies that shed data (Drop Oldest, Drop Newest, Adaptive Decimation).

While the compiled pipelines are fast enough on average to process the incoming data at higher ingestion rates, micro-jitter stemming from system fluctuations causes the bounded queue to temporarily fill to its maximum capacity. Flow-control policies manage these short-lived fluctuations by taking advantage of the 1-second unbounded buffer's capacity — when the bounded queue is full, the bridge leaves the unprocessed frames in the unbounded queue until space is available. This successfully absorbs the fluctuation without the loss of data, but at the cost of a temporary increase in latency.

Conversely, load-shedding policies aggressively drop frames when temporary jitter occurs. Because they are designed to adhere to tem-

poral deadlines, they lack the flexibility to leverage the unbounded buffer's capacity no matter how temporary the increase in latency may be. While this almost guarantees that surviving frames are processed within the latency deadline (particularly the Drop Oldest policy, which evicts the oldest frame in favour of the newest), this may result in a significant loss of data even at ingestion rates far below what the pipeline can typically sustain.

Adaptive Decimation attempts to bridge the gap between the flow-control policies (Bounded Queue, Exponential Backoff) and the load-shedding policies (Drop Oldest, Drop Newest). By dynamically downsampling the data stream at a linearly increasing rate as the bounded queue approaches full capacity (after entering a predefined threshold), the policy reduces pressure while preserving temporal continuity of the surviving frames. Though this prevents sequential bursts of dropped frames characteristic of the Drop Oldest and Drop Newest policies, its mechanism to drop frames *before* saturation is *potentially* reached results in substantial data loss that is often unnecessary, particularly at ingestion rates far below the pipeline's maximum sustainable throughput.

At terminal saturation (e.g. load 7.0), the advantages are reversed. Because flow-control policies do not drop frames, the unbounded circular buffer fills to maximum capacity, pushing tail latencies beyond the deadline (see Figure 14). In contrast, by aggressively dropping frames, the load-shedding policies are able to adhere to the latency deadline even under unyielding load, proving that at higher rates of ingestion data preservation must be sacrificed to guarantee deadline adherence (Figure 18).

As shown in Figure 17, Drop Oldest dropped fewer frames overall than Drop Newest. Because Drop Newest simply rejects incoming frames when the bounded buffer is full, it avoids the computational overhead and extended mutex lock duration required by Drop Oldest to overwrite existing queue data, and so this result was unexpected. However, while the root cause of this discrepancy requires further investigation, the evaluation results reveal that the heavier queue-modification logic of Drop Oldest does not negatively impact the pipeline's overall retention rate.

Because Drop Oldest ejects stale frames in favour of the freshest data, it ensures that surviving frames maintain temporal relevance and adhere to the latency deadline. Conversely, because Drop Newest preserves existing frames, those frames continue to age in the queue, often exceeding the latency deadline and becoming temporally irrelevant before they are processed. Alternatively, Adaptive Decimation can be used to preserve temporal continuity if that is a pipeline priority, but this proactive shedding occurs at the expense of a significantly greater loss of data, even at ingestion rates below what the pipeline can typically sustain.

7.9. Load-Shedding Overhead

For the static load-shedding policies (Drop Oldest and Drop Newest), C++ was able to sustain a maximum load multiplier of 1.0 when using the unconstrained power mode, while Rust was able to sustain a marginally higher load of 1.5. However, when using Adaptive Decimation, though C++ was still able to sustain 1.0, Rust's throughput dropped to 1.0.

The static load-shedding policies are very simple, and mainly rely on memory manipulation to check the buffer, and (when using Drop Oldest) to overwrite the oldest frame and update the queue index. Rust's ability to sustain a higher load when using these policies suggests that it is more efficient at memory manipulation than C++.

Conversely, Adaptive Decimation relies heavily on mathematical calculations as it linearly scales the decimation ratio based on the current queue depth, and performs modulo arithmetic. Rust's proportional drop in performance reveals that it is less efficient at performing these calculations than C++. This is likely because for division operations (including modulo), Rust inserts more machine

instructions to check if the divisor is zero so that it can perform a controlled panic, while C++ is optimised to perform the division without any checks, leading to undefined behaviour if the divisor is zero.

7.10. Mutex Contention

In the baseline performance (Section 6.1), while both C++ and Rust achieved a maximum sustained load of 5.5 using Exponential Backoff, both compiled implementations were only able to achieve a maximum load of 4.0 when using the Bounded Queue policy.

When using Bounded Queue, a saturated queue forces the bridge thread into a tight spin-loop, continuously acquiring and releasing the queue's mutex to check capacity. This continuous locking creates severe lock contention (see Figure 26). Because the bridge thread instantly attempts to reacquire the lock, the inference thread frequently fails to acquire the mutex, preventing it from removing a frame and creating the space that the bridge is waiting for. This creates a bottleneck that throttles both languages equally, regardless of the underlying efficiency of their respective mutex implementations.

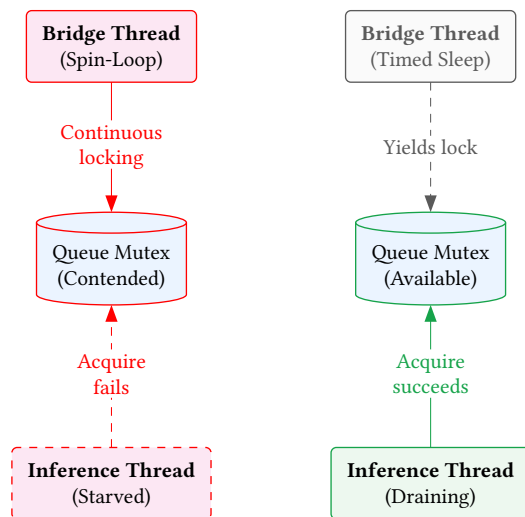


Figure 26: Comparison of mutex contention under queue saturation. On the left, Bounded Queue's continuous spin-loop causes severe lock contention, starving the inference thread. On the right, Exponential Backoff yields the lock, granting the consumer uncontested access to drain the backlog.

By forcing the bridge thread into a timed sleep when the queue is full, Exponential Backoff prevents the contention of the queue's mutex, increasing the inference thread's opportunity to remove a frame from the queue and make space for the bridge. Consequently, both C++ and Rust were able to achieve a higher ingestion rate for this flow-control policy.

7.11. The Python GIL and Concurrency

The data gathered in this report reveals poor performance when utilising CPython 3.10.12 using the unconstrained MAXN_SUPER power mode (see Section 6.1), Python was only able to sustain a load multiplier of 0.04 (i.e. 4% of the native sensor speed) when using the Exponential Backoff backpressure policy, and reached saturation point even at load multipliers of < 0.01 for all other policies, severely lagging behind the performance of the compiled languages.

Automated memory management via the GC was initially suspected to be the cause of this extreme latency. However, the GC overhead analysis graph (Figure 19) shows garbage collection pauses were confined to the 10-second initialisation window. Consequently, it could not be the cause of Python's deadline breaches during the steady-state phase.

Instead, the deadline breaches were likely caused by CPython's GIL. The HAR pipeline uses multiple threads (bridge, AI inference, late-

fusion, and telemetry) which would usually run concurrently across the multi-core CPU of the Jetson Orin Nano. However, the GIL is designed to prevent more than one thread from executing Python bytecode at any one time, effectively serialising the pipeline.

This was made worse by the use of Python's pass statement. C++ and Rust utilise micro-architectural CPU hints (`__asm__ volatile("yield" ::: "memory")` and `std::hint::spin_loop()` respectively) to pause speculative execution and yield resources without yielding to the OS kernel. Python lacks an equivalent mechanism, and instead empty pass spin-loops must be used — specifically at the IPC boundary when waiting for data from the unbounded buffer, in the inference thread when waiting for data from the bounded queue, and while waiting for space to be available in the bounded buffer (if the Bounded Queue policy is in use).

With three concurrent data streams, a total of six or nine pass spin-loops are used simultaneously depending on the active backpressure policy. These each aggressively hold the GIL, starving the other threads of execution time, including the threads responsible for fetching the data or making available space that the spin-loops are waiting for, and creating an effective deadlock. Because Exponential Backoff uses a timed sleep when the queue is full, it releases the GIL, granting the inference threads the opportunity to run. Conversely, the Bounded Queue policy forces the bridge thread into a continuous pass loop, retaining the lock and starving the remaining threads of execution time, resulting in underutilisation of resources and persistent breaches of the 100 ms deadline.

This GIL contention also explains the poor performance of the load-shedding policies (Drop Oldest, Drop Newest, and Adaptive Decimation). These policies never intentionally sleep or pause ingestion, and so consequently the bridge threads continuously evaluate thresholds and manipulate the queues, monopolising the GIL and preventing the inference threads from acquiring the lock long enough to process the surviving frames.

Furthermore, this GIL monopolisation also explains the thermal behaviour observed in Figure 24. At the lower, sustainable load of 0.04, Python generated heat faster than at the higher, native load of 1.0. This demonstrates that at unsustainable throughputs, the pipeline saturates almost instantly, forcing the bridge thread to apply the Exponential Backoff policy and repeatedly sleep, which reduces overall computational resource utilisation and slows heat generation. Conversely, at the lower, sustainable load of 0.04, the pipeline processes the incoming frames, and the bridge thread executes an empty pass spin-loop while waiting for new data. This continuous cycle of active execution and busy-waiting holds the GIL and constantly utilises the CPU, causing it to generate heat more rapidly.

7.12. Zero-Allocation & Performance (C++ vs Rust)

An implementation objective was to eliminate memory churn as a confounder when comparing the compiled runtime models. As demonstrated in Figure 20, this was successfully achieved with zero-allocation during the steady-state phase.

With memory allocation removed as a possible confounder, both compiled implementations achieved near-parity in baseline performance under moderate loads, easily satisfying the 100 ms latency deadline. Both also performed similarly at terminal saturation (load 7.0), with C++ processing 9.4% of frames within the 100 ms deadline, Rust processing 7.7% of frames within the deadline, and both languages converging to similar maximum tail latencies of 177.7 ms and 174.7 ms respectively. In contrast, Python's maximum latencies were far higher, proving that these are not ceiling limits imposed by the pipeline's buffer capacity, but rather evidence that both compiled languages share a performance limit in how quickly they can drain a saturated queue.

7.13. Resident Set Size (RSS)

Analysis of the RSS of each implementation (Figure 20) revealed that all three languages had memory footprints within ~50 MiB of each other. This suggests that the RSS can be mostly attributed to the shared AI dependencies (ONNX Runtime, CUDA, and TensorRT), rather than overhead of the runtime models themselves. Consequently, despite Python's interpreted nature and automated memory management, its memory overhead was only marginally higher (less than 7%) than both compiled languages, proving that in Edge-AI pipelines, the choice of runtime model has little impact on the memory footprint of the overall system.

The finding contradicts the conclusions of Pereira et al. [12], which ranked Rust as seventh in memory efficiency, trailing behind C++. However, that study was conducted when Rust utilised the `jemalloc` allocator. The results recorded for this dissertation reveal that following Rust's adoption of the standard system allocator by default (introduced in version 1.32.0), its memory footprint was slightly lower than that of C++ in this evaluation (by less than 1%). While this small difference may also be attributed to the newer ONNX Runtime C-API utilised by Rust, the data indicates that Rust now achieves memory footprint parity with C++.

7.14. Impact of Hardware Power Constraints

When running the evaluation using the constrained 7-Watt power mode, the compiled implementations suffered a reduction in performance when using the flow-control policies – both compiled languages dropped from 4.0 to 2.0 for Bounded Queue, and from 5.5 to 2.5 for Exponential Backoff. Conversely, the maximum sustainable load multiplier for the load-shedding policies (Drop Oldest, Drop Newest, and Adaptive Decimation) remained the same for both compiled languages as for the MAXN_SUPER baseline, with C++ sustaining 1.0 for all three policies, and Rust sustaining 1.5 for Drop Oldest and Drop Newest, and 1.0 for Adaptive Decimation. This suggests that the flow-control policies benefit from more computational resources, whereas the load-shedding policies are bounded by the latency deadline and micro-jitter, and so do not benefit from the unconstrained MAXN_SUPER power profile.

A small improvement was observed for all backpressure policies when running the Python implementation using the 7-Watt power profile. The maximum sustainable load multiplier for Bounded Queue increased from < 0.01 to 0.02, for Exponential Backoff from 0.04 to 0.06, and for all the load-shedding policies from < 0.01 to 0.01. This improvement in performance suggests that GIL thrashing occurs when more CPU cores are available. This happens when the GIL is released by one thread, and too many threads wake simultaneously to acquire it. When using MAXN_SUPER mode, four of the six cores are available to the pipeline. Consequently, when a thread's time slice ends and the GIL is released, the remaining three threads all compete for the GIL but only one can acquire it, forcing the other two to go back to sleep. This forces multiple context switches, the overhead of which consumes CPU-cycles that could otherwise be available to the pipeline. Conversely, when using the 7-Watt power profile only two cores are available to the pipeline, and so when the GIL is released, only one thread wakes to acquire it, reducing the context-switching overhead and improving the pipeline's throughput.

7.15. Static Analysis

Static code analysis was performed using the `lizard` complexity analyser to quantify the verbosity and complexity of the three functionally identical pipeline implementations. The analysis measured NLOC and CCN.

As shown in Table 2, the analysis revealed a higher NLOC count for the C++ implementation compared to Rust and Python. This reflects C++'s idiomatic boilerplate as the language mandates encapsulation, requiring header declarations, private members, and accessors and

mutators. Conversely, Python (which achieved the lowest NLOC count) relies on implied typing, and public-by-default attributes using naming conventions to indicate client access rights, reducing developer friction but shifting validation to runtime.

Pipeline Language	Total NLOC	Function Count	Average CCN	Max. CCN
C++	1,484	79	2.3	16
Rust	1,334	35	4.2	19
Python	1,026	41	3.2	19

Table 2: Static analysis metrics demonstrating idiomatic verbosity and complexity across the three pipeline implementations.

Interestingly, C++ had the lowest average CCN, while Rust had the highest. This is a limitation of using CCN to compare different programming paradigms. Rather than indicating that the C++ logic is simpler, this is due to C++'s verbosity. The average CCN is calculated by dividing the total complexity by the number of functions. Because C++ requires many trivial methods not necessary in the other languages, these reduce the average CCN result. For example, while a simple destructor to release memory in C++ increases the function count and thus reduces the average CCN, the same destructor is not required in Rust or Python (because of the automated resource management and garbage collection). Consequently, the average CCN for Rust and Python is artificially inflated by their lower function counts, despite the logical complexity of the overall code being lower.

While the boilerplate code skews the average CCN, examining the individual function metrics reveals the similarity between all three implementations. The primary execution loops responsible for the pipelines' main execution logic (e.g. `spawn_fusion_thread` and `spawn_telemetry_thread`) register the highest CCN results between 16 and 19, regardless of the language paradigm. This suggests that complexity of the logic is tied to the system design itself, and is not substantially changed by the choice of programming language.

8. Conclusion

8.1. Summary

This dissertation presents an analysis of three functionally identical HAR AI-Edge pipelines implemented in C++20, Rust 1.97.1, and CPython 3.10.12, and deployed to a resource-constrained NVIDIA Jetson Orin Nano platform. Empirical data was collected by executing the evaluation at increasingly high ingestion rates under various backpressure policies and power constraints. The collected data provided insight into how different memory management paradigms (manual, compiler-enforced, and automated), concurrency models, and backpressure policies interact to impact real-time performance, data preservation, and system stability.

8.2. Addressing the Research Questions

RQ1: Runtime Performance

The empirical data gathered during the evaluation revealed near-parity between the C++ and Rust implementations (Section 6.1). Both pipelines were implemented with zero-allocation and sustained ingestion rates up to a load multiplier of 5.5 with no dropped or lapped frames. At peak throughput, the RGB synchronisation anchor adhered to the 100 ms latency deadline, and the IMU buffers absorbed delays while awaiting late-fusion synchronisation.

A measured discrepancy between the compiled implementations was observed during load-shedding. While Rust sustained a higher maximum ingestion rate than C++ for the static Drop Oldest and Drop Newest policies, it lost this advantage when using Adaptive Decimation. Because Adaptive Decimation relies on division and

modulo arithmetic to dynamically downsample the data stream, it exposes a difference in how the compilers handle division operations. To ensure controlled panics in the event of a division-by-zero, Rust inserts additional machine instructions. Conversely, C++ is optimised to perform the division without these checks, gaining a small performance advantage at the risk of undefined behaviour. This small difference became a measurable discrepancy when using Adaptive Decimation, which performs division and modulo arithmetic thousands of times per second.

No garbage collection “stop-the-world” events occurred after the initial 10-second initialisation window in the Python implementation. However, due to contention with the automated memory management, a second shared memory buffer was required. The Python implementation only achieved a maximum sustainable load multiplier of 0.06 (i.e. 6% of the native sensor speed) when using the constrained 7-Watt power mode.

Analysis of the RSS (Figure 20) shows that all three implementations were within ~50 MiB of each other (approximately 745.6–793.8 MiB). This reveals that in Edge-AI deployments, the memory requirements are predominantly determined by shared AI dependencies (ONNX Runtime, CUDA, and TensorRT), rather than the language runtime models themselves.

RQ2: Backpressure Interaction

The discussion of backpressure policies (Section 7.8) identified a trade-off between data loss, temporal continuity, and deadline adherence. The flow-control policies successfully mitigated micro-jitter by using the capacity of the unbounded buffers, allowing the number of unprocessed frames to temporarily increase under moderate load, guaranteeing no data loss. However, as load increased, allowing unprocessed frames to accumulate pushed the tail latency up and breached the 100 ms deadline (Figure 14).

Conversely, the load-shedding policies dropped frames even at moderate loads, sacrificing data preservation and temporal continuity for deadline adherence. By dropping stale frames in favour of fresh data, Drop Oldest proved most effective at adhering to the latency deadline even under heavy loads.

If temporal continuity is a priority for the pipeline, Adaptive Decimation drops every n -th frame in an attempt to prevent total saturation, but does so at the expense of more dropped frames even during brief periods of moderate load or micro-jitter that the pipeline would be otherwise able to sustain without data loss.

By isolating the saturation points of the individual streams under the Exponential Backoff policy (Figure 15), it was shown that synchronisation anchors (such as the RGB frames during late-fusion) bottleneck upstream processing, dictating the pipeline’s overall capacity.

The choice of backpressure policy significantly impacted the concurrency overhead. Exponential Backoff yielded to the kernel when the bounded queue was full, allowing other threads to drain the queue and make space for the bridge thread. However, the remaining policies all rely on continuous loops without yielding, and cause severe mutex contention (Section 7.10). In particular, Bounded Queue acquires the bounded buffer’s mutex in a tight spin-loop while waiting for space to become available. This creates contention with the inference thread as it attempts to acquire the same mutex to drain the queue, resulting in resource starvation that acts as a livelock. Due to the GIL preventing multiple threads from executing Python bytecode simultaneously, this concurrent contention of mutex locks was particularly severe for the Python implementation.

Because the pipelines use pre-allocated memory, the zero-allocation design prevented the backpressure policies from causing dynamic memory churn or GC pauses during steady-state processing.

RQ3: Dynamic Profiling vs. Runtime Behaviour

Statistical analysis, using Kruskal-Wallis and Spearman’s rank correlation, was effective in diagnosing runtime behaviour. Performing a Kruskal-Wallis H-test confirmed that performance was related to the runtime models, and not random system noise. When using Exponential Backoff, Dunn’s post-hoc pairwise comparison with a Bonferroni correction (Table 1) revealed that there was no significant difference between C++ and Rust, but that the Python implementation was slower due to the runtime model itself.

Spearman’s rank correlation revealed that there was no relationship between CPU temperature and latency. Because the Jetson Orin Nano’s thermal management triggered the cooling fan at 74°C, DVFS throttling did not occur. The analysis results confirmed that performance degradation of the compiled implementations when using the constrained 7-Watt power mode (Section 6.9), instead of the unconstrained MAXN_SUPER mode, was a result of reduced computational resources (i.e. a reduced number of CPU cores and lower clock frequencies).

An undefined result (NaN) was returned when calculating Spearman’s rank correlation (ρ) for the impact of Python’s GC pauses on latency, confirming that no GC events took place in the pipeline after the 10-second initialisation window. Similarly, an undefined result was also returned when calculating ρ for the impact of dynamic memory allocation (in the C++ and Rust implementations) on CPU temperature, confirming that no memory was dynamically allocated during the steady-state phase of the evaluation.

8.3. Recommendations for Edge-AI Deployments

Based on the findings of this dissertation, Rust is highly recommended for multi-threaded, real-time pipelines deployed to Edge-AI platforms such as the Jetson Orin Nano. It offers the zero-allocation performance and execution speed of C++, while swapping developer discipline and vigilance for compiler-enforced memory safety, reducing intermittent memory-safety bugs and the long-term maintenance overhead of complex concurrent systems. However, the novel ownership model, the “zerover” pre-release states of some Rust crates, and conflicting online documentation, may increase the learning curve for developers unfamiliar with the language and its ecosystem.

CPython’s reduced NLOC and eradication of compile-time overhead may make it suitable for initial pipeline prototyping. However, contrary to common assumption, its automatic memory management may increase developer friction when employed for systems-engineering tasks in a multi-threaded architecture. In addition, it demonstrated concurrency limitations for high-speed ingestion systems. This is due to both the GIL, which restricts the number of threads actively executing Python bytecode to just one, and the absence of a CPU micro-architectural hint to yield resources without yielding to the OS kernel.

When selecting a backpressure policy for a high-speed pipeline, the architect must decide between data preservation, temporal relevance, and temporal continuity. Flow-control policies (Bounded Queue and Exponential Backoff) should be considered if the preservation of data is the priority and computational resources are sufficient for the expected maximum load. However, if temporal relevance is the priority, the Drop Oldest load-shedding policy ejects stale frames in favour of fresh data when the data ingestion rate exceeds the pipeline’s ability to process data within the latency deadline. Finally, if temporal continuity is the priority, Adaptive Decimation proactively drops every n -th frame before saturation is reached, at the expense of a significantly higher drop rate as the data ingestion rate approaches the pipeline’s maximum sustainable throughput.

8.4. Security, Ethical, and Professional Considerations

System security can be compromised by memory-safety bugs, such as out-of-bounds and use-after-free vulnerabilities, which can be intermittent and difficult to replicate in testing. By evaluating the manual memory management of C++, the compiler-enforced memory safety of Rust, and the automated memory management of Python, this dissertation addresses the professional need to reduce the deployment of vulnerable code to production environments.

Processing sensitive data (such as video) locally on Edge-AI devices addresses social and ethical privacy concerns. Pipelines such as the one presented in this dissertation eliminate the requirement to transmit data to remote cloud servers. This prevents network interception, and the legal risks associated with data breaches and the mishandling of personal information.

Edge-AI hardware places resource constraints on deployments, such as available power and thermal dissipation limits. By evaluating runtime models under a constrained 7-Watt power profile, and analysing the impact of thermal accumulation on performance, this dissertation provides guidance for the professional implementation of resource-efficient pipelines.

8.5. Future Work

While this dissertation evaluated the interaction between runtime models and backpressure policies, several areas of future investigative work have been identified:

1. **GIL Profiling and Truly Concurrent Python:** Having eliminated Garbage Collection as the cause of Python's poor maximum sustainable performance, future work might investigate the impact of the GIL and its serialisation of thread execution, alongside the consequential thread contention. A low-level profiling tool could be used to provide more visibility into thread starvation. Furthermore, alternative Python implementations, such as the PyPy-STM (Software Transactional Memory) variant, could determine if Python can achieve parity with compiled languages in real-time Edge-AI pipelines.
2. **Fanless Platforms:** The active cooling of the Jetson Orin Nano Developer Kit automatically starts the fan when the temperature reaches 74°C, preventing the system from reaching its 99°C DVFS threshold. Future work should evaluate these pipelines when deployed to passively cooled platforms. Forcing the hardware to reach thermal limits would provide visibility into how the implementations interact with dynamic thermal throttling when under sustained load.
3. **Heterogeneous Backpressure:** Asymmetrical saturation was identified on the RGB stream when using Exponential Backoff, caused by the late-fusion synchronisation anchor. Future research could evaluate the impact of using different backpressure policies on the anchored RGB stream and the two flexible IMU streams. Applying a load-shedding policy to the RGB stream could guarantee latency deadline adherence, while applying a flow-control policy to the IMU streams could maximise data preservation. Using a heterogeneous backpressure approach may offer a balance between pipeline stability and prediction accuracy.
4. **Load-Shedding Efficiency Discrepancy:** Unexpectedly, the Drop Oldest policy dropped fewer frames than Drop Newest. Drop Oldest executes more instructions to overwrite the oldest frame while holding the queue's mutex lock, whereas Drop Newest simply rejects incoming frames. It was therefore expected that Drop Newest would be more efficient and drop fewer frames, but the data showed the opposite. Future profiling work should investigate the cause of this discrepancy to determine why simply rejecting a new frame is outperformed by a policy that must manage the bounded buffer.

Appendix A: Online Access and Reproducibility

The following files and scripts are publicly available for review:

- Deterministic load generator:
 - `code/generator/`
- Complete source code for the three pipeline implementations:
 - `code/pipeline-cpp/` (C++20)
 - `code/pipeline-py/` (CPython 3.10.12)
 - `code/pipeline-rust/` (Rust 1.97.1)
- Docker deployment environment:
 - `Dockerfile`
- Shell scripts used to deploy the suite and evaluate the results:
 - `compile_trt.py` and `torch/generate.py` (compile the ONNX models)
 - `deploy.sh` (deploys the evaluation suite to the Jetson Orin Nano)
 - `run.sh` (executes the evaluation suite on the Jetson Orin Nano)
 - `pull_results.sh` (downloads the results to the host machine)
 - `log2csv.sh` (creates `.csv` files from the raw `.log` files)
- Jupyter Notebook to analyse the results and generate the figures used in this report:
 - `results/analysis.ipynb`

Running `deploy.sh` from the directory containing the script, with the Jetson Orin Nano connected to the host machine via USB, will deploy the evaluation suite using the `Dockerfile` to create a reproducible environment.

After logging directly into the Jetson Orin Nano, or connecting to it via SSH, the evaluation suite can be executed using the `run.sh` script in the `~/dissertation` directory. The script will balance the hardware temperatures, execute the evaluation suite, and run `tegrastats`:

- `run.sh 2` will execute the evaluation suite in MAXN_SUPER mode
- `run.sh 3` will execute the evaluation suite in constrained 7-Watt mode

Note that the Jetson will need to be rebooted after switching between power modes.

After completion of the evaluation suite, the results can be downloaded via USB to the host machine using the `pull_results.sh` script. The results will be stored in a subdirectory of the `results/` directory, automatically named with the current date and time of the evaluation. The raw `.log` files can be converted to `.csv` format for analysis using the `log2csv.sh` script, which must be followed by the directory name of the `.log` files to be converted. The downloaded results directory must be named `07-watt` or `MAXN_SUPER` as appropriate for analysis.

Jupyter Lab can be started from the `results/` directory using the command `poetry run jupyter lab`. Note that `poetry` must be installed on the host machine. The `analysis.ipynb` notebook can then be opened to analyse the results and generate the figures used in this report.

Source Code Repository: <https://github.com/timclarke76/dissertation>

Release Tag: `v1.0.0-submission`

Due to the volume of the telemetry logs, the raw `.csv` and `.log` files are hosted on the cloud. They can be accessed and downloaded via the following link:

https://dmail-my.sharepoint.com/:f/g/personal/2712139_dundee_ac_uk/IgBd2GmOn1j3R6u7qx_Kip9uAQz75AvBoju15ezPvIkyaE?e=4cGkgX

Appendix B: Hardware Power Profiles

Raw configuration output from the Jetson Orin Nano nvpmode daemon, detailing core availability, and minimum and maximum caps for all power modes.

```
NVPM VERB: Config file: /etc/nvpmode.conf
NVPM VERB: parsing done for /etc/nvpmode.conf
succeed to parse file /etc/nvpmode.conf.
NVPM VERB: PM_CONFIG: DEFAULT=25W(1)
NVPM VERB: PARAM TYPE=FILE NAME=CPU_ONLINE
NVPM VERB: CORE_0 /sys/devices/system/cpu/cpu0/online
NVPM VERB: CORE_1 /sys/devices/system/cpu/cpu1/online
NVPM VERB: CORE_2 /sys/devices/system/cpu/cpu2/online
NVPM VERB: CORE_3 /sys/devices/system/cpu/cpu3/online
NVPM VERB: CORE_4 /sys/devices/system/cpu/cpu4/online
NVPM VERB: CORE_5 /sys/devices/system/cpu/cpu5/online
NVPM VERB: PARAM TYPE=FILE NAME=FBP_POWER_GATING
NVPM VERB: FBP_PG_MASK /sys/devices/platform/gpu.0/fbp_pg_mask
NVPM VERB: PARAM TYPE=FILE NAME=TPC_POWER_GATING
NVPM VERB: TPC_PG_MASK /sys/devices/platform/gpu.0/tpc_pg_mask
NVPM VERB: PARAM TYPE=FILE NAME=GPU_POWER_CONTROL_ENABLE
NVPM VERB: GPU_PWR_CNTL_EN /sys/devices/platform/gpu.0/power/control
NVPM VERB: PARAM TYPE=FILE NAME=GPU_POWER_CONTROL_DISABLE
NVPM VERB: GPU_PWR_CNTL_DIS /sys/devices/platform/gpu.0/power/control
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_0
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_1
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu1/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu1/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu1/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_2
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu2/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu2/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu2/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_3
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu3/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu3/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu3/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_4
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu4/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu4/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu4/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=CPU_A78_5
NVPM VERB: FREQ_TABLE /sys/devices/system/cpu/cpu5/cpufreq/scaling_available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/system/cpu/cpu5/cpufreq/scaling_max_freq
NVPM VERB: MIN_FREQ /sys/devices/system/cpu/cpu5/cpufreq/scaling_min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=GPU
NVPM VERB: FREQ_TABLE /sys/devices/platform/17000000.gpu/devfreq_dev/available_frequencies
NVPM VERB: MAX_FREQ /sys/devices/platform/17000000.gpu/devfreq_dev/max_freq
NVPM VERB: MIN_FREQ /sys/devices/platform/17000000.gpu/devfreq_dev/min_freq
NVPM VERB: PARAM TYPE=CLOCK NAME=EMC
NVPM VERB: MAX_FREQ /sys/kernel/nvpmode_clk_cap/emc
NVPM VERB:
NVPM VERB: POWER_MODEL: ID=0 NAME=15W
NVPM VERB: CPU_ONLINE CORE_0 1
NVPM VERB: CPU_ONLINE CORE_1 1
NVPM VERB: CPU_ONLINE CORE_2 1
NVPM VERB: CPU_ONLINE CORE_3 1
NVPM VERB: CPU_ONLINE CORE_4 1
NVPM VERB: CPU_ONLINE CORE_5 1
NVPM VERB: FBP_POWER_GATING FBP_PG_MASK 2
NVPM VERB: TPC_POWER_GATING TPC_PG_MASK 240
NVPM VERB: GPU_POWER_CONTROL_ENABLE GPU_PWR_CNTL_EN on
NVPM VERB: CPU_A78_0 MIN_FREQ 729600
NVPM VERB: CPU_A78_0 MAX_FREQ 1497600
NVPM VERB: CPU_A78_1 MIN_FREQ 729600
NVPM VERB: CPU_A78_1 MAX_FREQ 1497600
NVPM VERB: CPU_A78_2 MIN_FREQ 729600
NVPM VERB: CPU_A78_2 MAX_FREQ 1497600
NVPM VERB: CPU_A78_3 MIN_FREQ 729600
NVPM VERB: CPU_A78_3 MAX_FREQ 1497600
NVPM VERB: CPU_A78_4 MIN_FREQ 729600
NVPM VERB: CPU_A78_4 MAX_FREQ 1497600
NVPM VERB: CPU_A78_5 MIN_FREQ 729600
NVPM VERB: CPU_A78_5 MAX_FREQ 1497600
NVPM VERB: GPU_MIN_FREQ 0
NVPM VERB: GPU_MAX_FREQ 612000000
NVPM VERB: GPU_POWER_CONTROL_DISABLE GPU_PWR_CNTL_DIS auto
NVPM VERB: EMC_MAX_FREQ 2133000000
NVPM VERB:
NVPM VERB: POWER_MODEL: ID=1 NAME=25W
NVPM VERB: CPU_ONLINE CORE_0 1
NVPM VERB: CPU_ONLINE CORE_1 1
NVPM VERB: CPU_ONLINE CORE_2 1
NVPM VERB: CPU_ONLINE CORE_3 1
NVPM VERB: CPU_ONLINE CORE_4 1
NVPM VERB: CPU_ONLINE CORE_5 1
NVPM VERB: FBP_POWER_GATING FBP_PG_MASK 2
NVPM VERB: TPC_POWER_GATING TPC_PG_MASK 240
NVPM VERB: GPU_POWER_CONTROL_ENABLE GPU_PWR_CNTL_EN on
NVPM VERB: CPU_A78_0 MIN_FREQ 729600
NVPM VERB: CPU_A78_0 MAX_FREQ 1344000
NVPM VERB: CPU_A78_1 MIN_FREQ 729600
NVPM VERB: CPU_A78_1 MAX_FREQ 1344000
```

```
NVPM VERB: CPU_A78_2 MIN_FREQ 729600
NVPM VERB: CPU_A78_2 MAX_FREQ 1344000
NVPM VERB: CPU_A78_3 MIN_FREQ 729600
NVPM VERB: CPU_A78_3 MAX_FREQ 1344000
NVPM VERB: CPU_A78_4 MIN_FREQ 729600
NVPM VERB: CPU_A78_4 MAX_FREQ 1344000
NVPM VERB: CPU_A78_5 MIN_FREQ 729600
NVPM VERB: CPU_A78_5 MAX_FREQ 1344000
NVPM VERB: GPU_MIN_FREQ 0
NVPM VERB: GPU_MAX_FREQ 918000000
NVPM VERB: GPU_POWER_CONTROL_DISABLE GPU_PWR_CNTL_DIS auto
NVPM VERB: EMC_MAX_FREQ 3199000000
NVPM VERB:
NVPM VERB: POWER_MODEL: ID=2 NAME=MAXN_SUPER
NVPM VERB: CPU_ONLINE CORE_0 1
NVPM VERB: CPU_ONLINE CORE_1 1
NVPM VERB: CPU_ONLINE CORE_2 1
NVPM VERB: CPU_ONLINE CORE_3 1
NVPM VERB: CPU_ONLINE CORE_4 1
NVPM VERB: CPU_ONLINE CORE_5 1
NVPM VERB: FBP_POWER_GATING FBP_PG_MASK 2
NVPM VERB: TPC_POWER_GATING TPC_PG_MASK 240
NVPM VERB: GPU_POWER_CONTROL_ENABLE GPU_PWR_CNTL_EN on
NVPM VERB: CPU_A78_0 MIN_FREQ 729600
NVPM VERB: CPU_A78_0 MAX_FREQ 9223372036854775807
NVPM VERB: CPU_A78_1 MIN_FREQ 729600
NVPM VERB: CPU_A78_1 MAX_FREQ 9223372036854775807
NVPM VERB: CPU_A78_2 MIN_FREQ 729600
NVPM VERB: CPU_A78_2 MAX_FREQ 9223372036854775807
NVPM VERB: CPU_A78_3 MIN_FREQ 729600
NVPM VERB: CPU_A78_3 MAX_FREQ 9223372036854775807
NVPM VERB: CPU_A78_4 MIN_FREQ 729600
NVPM VERB: CPU_A78_4 MAX_FREQ 9223372036854775807
NVPM VERB: CPU_A78_5 MIN_FREQ 729600
NVPM VERB: CPU_A78_5 MAX_FREQ 9223372036854775807
NVPM VERB: GPU_MIN_FREQ 0
NVPM VERB: GPU_MAX_FREQ 9223372036854775807
NVPM VERB: GPU_POWER_CONTROL_DISABLE GPU_PWR_CNTL_DIS auto
NVPM VERB: EMC_MAX_FREQ 9223372036854775807
NVPM VERB:
NVPM VERB: POWER_MODEL: ID=3 NAME=7W
NVPM VERB: CPU_ONLINE CORE_0 1
NVPM VERB: CPU_ONLINE CORE_1 1
NVPM VERB: CPU_ONLINE CORE_2 1
NVPM VERB: CPU_ONLINE CORE_3 1
NVPM VERB: CPU_ONLINE CORE_4 0
NVPM VERB: CPU_ONLINE CORE_5 0
NVPM VERB: FBP_POWER_GATING FBP_PG_MASK 2
NVPM VERB: TPC_POWER_GATING TPC_PG_MASK 252
NVPM VERB: GPU_POWER_CONTROL_ENABLE GPU_PWR_CNTL_EN on
NVPM VERB: CPU_A78_0 MIN_FREQ 729600
NVPM VERB: CPU_A78_0 MAX_FREQ 960000
NVPM VERB: GPU_MIN_FREQ 0
NVPM VERB: GPU_MAX_FREQ 408000000
NVPM VERB: GPU_POWER_CONTROL_DISABLE GPU_PWR_CNTL_DIS auto
NVPM VERB: EMC_MAX_FREQ 2133000000
NVPM VERB:
```

Appendix C: Hardware Thermal Configurations

Raw configuration output from the Jetson Linux `sysfs` thermal zones, detailing the hardware trip points for active cooling, software thermal throttling (DVFS), and critical hardware shutdowns. Temperatures are represented in millidegrees Celsius.

```
tim@jetson:~$ grep "" /sys/class/thermal/thermal_zone*/type
/sys/class/thermal/thermal_zone0/type:cpu-thermal
/sys/class/thermal/thermal_zone1/type:gpu-thermal
/sys/class/thermal/thermal_zone2/type:cv0-thermal
/sys/class/thermal/thermal_zone3/type:cv1-thermal
/sys/class/thermal/thermal_zone4/type:cv2-thermal
/sys/class/thermal/thermal_zone5/type:soc0-thermal
/sys/class/thermal/thermal_zone6/type:soc1-thermal
/sys/class/thermal/thermal_zone7/type:soc2-thermal
/sys/class/thermal/thermal_zone8/type:tj-thermal

tim@jetson:~$ grep "" /sys/class/thermal/thermal_zone*/trip_point*_temp
/sys/class/thermal/thermal_zone0/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone0/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone0/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone1/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone1/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone1/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone2/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone2/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone2/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone3/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone3/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone3/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone4/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone4/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone4/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone5/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone5/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone5/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone6/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone6/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone6/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone7/trip_point_0_temp:99000
/sys/class/thermal/thermal_zone7/trip_point_1_temp:104500
/sys/class/thermal/thermal_zone7/trip_point_2_temp:70000
/sys/class/thermal/thermal_zone8/trip_point_0_temp:35000
/sys/class/thermal/thermal_zone8/trip_point_1_temp:74000
/sys/class/thermal/thermal_zone8/trip_point_2_temp:95000
/sys/class/thermal/thermal_zone8/trip_point_3_temp:104500
```

Appendix D: Static Code Analysis Raw Output

The following raw output was generated by the lizard static code analyser, detailing the Non-commenting Lines Of Code (NLOC) and Cyclomatic Complexity Number (CCN) for every function across the three pipeline implementations.

D.1 C++20

NLOC	CCN	token	PARAM	length	location
9	1	75	2	11	Args::Args@3-13@src/config/args.cpp
1	1	9	0	1	Args::settings@25-25@src/config/args.h
4	1	9	0	4	Settings::get_rgb_queue_config@49-52@src/config/settings.h
1	1	9	0	1	Settings::get_rgb_policy@57-57@src/config/settings.h
4	1	9	0	4	Settings::get_accel_queue_config@62-65@src/config/settings.h
1	1	9	0	1	Settings::get_accel_policy@70-70@src/config/settings.h
4	1	9	0	4	Settings::get_gyro_queue_config@75-78@src/config/settings.h
1	1	9	0	1	Settings::get_gyro_policy@83-83@src/config/settings.h
17	2	121	1	21	Settings::Settings@5-25@src/config/settings.cpp
44	8	346	2	50	Settings::parse_event_queue_config@28-77@src/config/settings.cpp
35	8	331	2	40	Settings::parse_policy@80-119@src/config/settings.cpp
8	1	68	2	8	TelemetryWriter::TelemetryWriter@146-153@src/thread/telemetry.h
8	2	33	0	9	TelemetryWriter::terminate@176-184@src/thread/telemetry.h
121	16	853	2	163	spawn_fusion_thread@10-172@src/thread/fusion.cpp
14	2	66	0	15	TelemetryWriter::Epoch::reset@15-29@src/thread/telemetry.cpp
24	3	121	1	29	TelemetryWriter::Csv::Csv@31-59@src/thread/telemetry.cpp
6	2	22	0	6	TelemetryWriter::Csv::~Csv@61-66@src/thread/telemetry.cpp
37	4	289	2	49	TelemetryWriter::Csv::write_epoch@69-117@src/thread/telemetry.cpp
24	4	180	3	29	TelemetryWriter::record@120-148@src/thread/telemetry.cpp
9	2	63	0	12	TelemetryWriter::swap_buffers@151-162@src/thread/telemetry.cpp
79	12	580	3	114	spawn_telemetry_thread@165-278@src/thread/telemetry.cpp
94	15	613	4	152	spawn_bridge_thread@19-170@src/thread/bridge.cpp
66	8	546	7	86	spawn_inference_thread@12-97@src/thread/inference.cpp
6	1	38	0	6	current_time_nanos@10-15@src/os/time.h
1	1	9	0	1	spin_loop@10-10@src/os/os.h
1	1	14	0	1	spin_loop@12-12@src/os/os.h
1	1	5	0	1	spin_loop@14-14@src/os/os.h
5	2	41	2	5	saturating_sub@29-33@src/os/os.h
5	1	18	1	5	ChannelState::ChannelState@20-24@src/os/channel.h
4	1	20	1	4	Sender::Sender@37-40@src/os/channel.h
4	1	21	1	4	Sender::Sender@44-47@src/os/channel.h
7	2	34	1	8	Sender::operator=@51-58@src/os/channel.h
5	1	30	1	5	Sender::send@71-75@src/os/channel.h
4	1	20	1	4	Receiver::Receiver@93-96@src/os/channel.h
4	1	21	1	4	Receiver::Receiver@100-103@src/os/channel.h
7	2	34	1	8	Receiver::operator=@107-114@src/os/channel.h
7	1	28	0	7	Receiver::receive@123-129@src/os/channel.h
9	2	37	0	11	Receiver::try_receive@138-148@src/os/channel.h
5	1	42	1	5	make_channel@164-168@src/os/channel.h
5	1	14	0	5	telemetry::ScopedToggle::ScopedToggle@17-21@src/os/allocator.h
1	1	9	0	1	telemetry::ScopedToggle::~ScopedToggle@23-23@src/os/allocator.h
11	3	69	1	13	operator new@21-33@src/os/allocator.cpp
4	1	19	1	4	operator new []@40-43@src/os/allocator.cpp
7	2	43	2	8	operator delete@52-59@src/os/allocator.cpp
4	1	25	2	4	operator delete []@66-69@src/os/allocator.cpp
4	1	17	1	4	operator delete@77-80@src/os/allocator.cpp
4	1	19	1	4	operator delete []@88-91@src/os/allocator.cpp
13	3	109	2	17	operator new@102-118@src/os/allocator.cpp
4	1	26	2	4	operator new []@127-130@src/os/allocator.cpp
9	2	53	3	10	operator delete@141-150@src/os/allocator.cpp
4	1	32	3	4	operator delete []@159-162@src/os/allocator.cpp
21	3	136	2	25	ShmBuffer::ShmBuffer@15-39@src/os/shm.cpp
34	5	220	0	55	ShmBuffer::next_frame@42-96@src/os/shm.cpp
25	4	180	1	32	ShmBuffer::open_shm_file@99-130@src/os/shm.cpp
15	1	77	1	47	ShmBuffer::aligns@15-61@src/os/shm.h
52	2	342	3	60	InferenceEngine::InferenceEngine@10-69@src/inference/inference_engine.cpp
11	2	57	0	11	InferenceEngine::run@72-82@src/inference/inference_engine.cpp
11	3	48	1	13	Histogram::operator=@4-16@src/histogram.cpp
8	2	49	3	8	Histogram::Histogram@17-24@src/histogram.h
6	2	16	0	6	Histogram::~Histogram@27-32@src/histogram.h
5	1	22	1	5	Histogram::Histogram@37-41@src/histogram.h
1	1	14	1	1	Histogram::record@56-56@src/histogram.h
1	1	10	0	1	Histogram::reset@59-59@src/histogram.h
4	1	17	1	4	Histogram::value_at_percentile@65-68@src/histogram.h
1	1	12	0	1	Histogram::max@73-73@src/histogram.h
5	1	21	1	5	Queue::Queue@17-21@src/queue.h
10	2	39	1	12	Queue::try_push@30-41@src/queue.h
6	1	29	1	6	Queue::overwrite_oldest@48-53@src/queue.h
10	2	51	0	12	Queue::pop@60-71@src/queue.h
1	1	9	0	1	Queue::len@75-75@src/queue.h
1	1	9	0	1	Queue::capacity@79-79@src/queue.h
1	1	12	1	1	Queue::increment_lapped_frames@83-83@src/queue.h
1	1	9	0	1	Queue::get_lapped_frames@87-87@src/queue.h
4	1	14	1	4	Queue::increment_dropped_frames@91-94@src/queue.h
1	1	9	0	1	Queue::get_dropped_frames@99-99@src/queue.h
1	1	8	0	1	Queue::get_mutex@103-103@src/queue.h
1	1	17	1	1	Queue::advance_index@109-109@src/queue.h
27	2	163	8	28	spawn_bridge_and_inference_threads@53-80@src/main.cpp
48	4	342	2	61	main@83-143@src/main.cpp
26 file analyzed.					
NLOC	Avg.NLOC	Avg.CCN	Avg.token	function_cnt	file
10	9.0	1.0	75.0	1	src/config/args.cpp
10	1.0	1.0	9.0	1	src/config/args.h

25	0.0	0.0	0.0	0	src/config/policy.h
45	2.5	1.0	9.0	6	src/config/settings.h
100	32.0	6.0	266.0	3	src/config/settings.cpp
8	0.0	0.0	0.0	0	src/thread/fusion.h
12	0.0	0.0	0.0	0	src/thread/bridge.h
13	0.0	0.0	0.0	0	src/thread/inference.h
86	8.0	1.5	50.5	2	src/thread/telemetry.h
128	121.0	16.0	853.0	1	src/thread/fusion.cpp
208	27.6	4.1	188.7	7	src/thread/telemetry.cpp
104	94.0	15.0	613.0	1	src/thread/bridge.cpp
74	66.0	8.0	546.0	1	src/thread/inference.cpp
9	6.0	1.0	38.0	1	src/os/time.h
13	2.0	1.2	17.2	4	src/os/os.h
90	5.5	1.3	27.7	11	src/os/channel.h
20	3.0	1.0	11.5	2	src/os/allocator.h
83	6.4	1.6	41.2	10	src/os/allocator.cpp
93	26.7	4.0	178.7	3	src/os/shm.cpp
58	15.0	1.0	77.0	1	src/os/shm.h
18	0.0	0.0	0.0	0	src/inference/inference_engine.h
70	31.5	2.0	199.5	2	src/inference/inference_engine.cpp
13	11.0	3.0	48.0	1	src/histogram.cpp
37	3.7	1.3	20.0	7	src/histogram.h
60	3.5	1.2	18.9	12	src/queue.h
97	37.5	3.0	252.5	2	src/main.cpp

=====
 !!!! Warnings (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100) !!!!
 =====

NLOC	CCN	token	PARAM	length	location

121	16	853	2	163	spawn_fusion_thread@10-172@src/thread/fusion.cpp

=====

Total nloc	Avg.NLOC	AvgCCN	Avg.token	Fun Cnt	Warning cnt	Fun Rt	nloc Rt

1484	13.5	2.3	90.5	79	1	0.01	0.11

D.2 Rust 1.97.1

=====						
NLOC	CCN	token	PARAM	length	location	

117	14	599	4	180	spawn_bridge_thread@27-206@./src/thread/bridge.rs	
130	9	695	2	168	spawn_fusion_thread@24-191@./src/thread/fusion.rs	
95	8	519	7	118	spawn_inference_thread@35-152@./src/thread/inference.rs	
36	7	163	0	42	try_new@87-128@./src/thread/telemetry.rs	
13	2	75	1	14	reset@131-144@./src/thread/telemetry.rs	
4	1	33	0	4	create_histogram@152-155@./src/thread/telemetry.rs	
17	3	79	1	20	try_new@173-192@./src/thread/telemetry.rs	
36	3	290	3	45	write_epoch@201-245@./src/thread/telemetry.rs	
17	2	72	2	18	try_new@295-312@./src/thread/telemetry.rs	
33	7	218	4	39	record@327-365@./src/thread/telemetry.rs	
10	2	77	1	20	swap_buffers@370-389@./src/thread/telemetry.rs	
13	3	89	1	18	terminate@397-414@./src/thread/telemetry.rs	
99	19	598	3	134	spawn_telemetry_thread@431-564@./src/thread/telemetry.rs	
5	2	51	0	6	now_nanos@9-14@./src/os/time.rs	
5	1	44	0	6	new@23-28@./src/os/allocator.rs	
3	1	26	1	3	drop@32-34@./src/os/allocator.rs	
8	2	77	2	14	alloc@39-52@./src/os/allocator.rs	
7	2	70	3	11	dealloc@55-65@./src/os/allocator.rs	
3	1	17	1	3	new@88-90@./src/os/shm.rs	
34	4	193	2	38	try_new@216-253@./src/os/shm.rs	
59	6	368	1	84	next_frame@270-353@./src/os/shm.rs	
26	9	199	1	31	open_shm_file@365-395@./src/os/shm.rs	
12	4	80	1	15	try_new@60-74@./src/config/settings.rs	
57	14	418	2	63	try_new@32-94@./src/inference/inference_engine.rs	
5	1	32	1	5	input_buffer_mut@101-105@./src/inference/inference_engine.rs	
21	4	112	1	26	run@112-137@./src/inference/inference_engine.rs	
25	1	145	8	28	spawn_bridge_and_inference_threads@49-76@./src/main.rs	
83	6	482	0	98	main@78-175@./src/main.rs	
13	2	59	1	14	new@34-47@./src/queue.rs	
9	2	75	2	10	try_push@56-65@./src/queue.rs	
5	1	53	2	8	overwrite_oldest@72-79@./src/queue.rs	
9	2	61	1	11	pop@85-95@./src/queue.rs	
3	1	13	1	3	len@101-103@./src/queue.rs	
3	1	13	1	3	capacity@109-111@./src/queue.rs	
3	1	23	2	3	advance_index@119-121@./src/queue.rs	

17 file analyzed.

=====

NLOC	Avg.NLOC	AvgCCN	Avg.token	function_cnt	file

8	0.0	0.0	0.0	0	./src/thread/mod.rs
129	117.0	14.0	599.0	1	./src/thread/bridge.rs
137	130.0	9.0	695.0	1	./src/thread/fusion.rs
106	95.0	8.0	519.0	1	./src/thread/inference.rs
333	27.8	4.9	169.4	10	./src/thread/telemetry.rs
7	5.0	2.0	51.0	1	./src/os/time.rs
6	0.0	0.0	0.0	0	./src/os/mod.rs
42	5.8	1.5	54.2	4	./src/os/allocator.rs
197	30.5	5.0	194.2	4	./src/os/shm.rs
6	0.0	0.0	0.0	0	./src/config/mod.rs
18	0.0	0.0	0.0	0	./src/config/policy.rs
35	12.0	4.0	80.0	1	./src/config/settings.rs
15	0.0	0.0	0.0	0	./src/config/args.rs
2	0.0	0.0	0.0	0	./src/inference/mod.rs
103	27.7	6.3	187.3	3	./src/inference/inference_engine.rs
131	54.0	3.5	313.5	2	./src/main.rs
59	6.4	1.4	42.4	7	./src/queue.rs

```

=====
!!!! Warnings (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100) !!!!
=====
NLOC   CCN   token  PARAM  length  location
-----
    99    19    598     3    134 spawn_telemetry_thread@431-564@./src/thread/telemetry.rs
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
   1334    29.1    4.2    174.8    35        1    0.03   0.10
=====

```

D.3 CPython 3.10.12

```

=====
NLOC   CCN   token  PARAM  length  location
-----
     2     2     20     2     14 saturating_sub@1-14@include/os/os.py
     6     3     42     2     9  gc_callback@9-17@include/os/gc.py
    16     1     85     7    69  __init__@61-129@include/os/shm.py
    23     4    137     3    40  __init__@198-237@include/os/shm.py
     3     1     18     1     4  close@239-242@include/os/shm.py
    30     5    181     1    68  next_frame@244-311@include/os/shm.py
     2     1     29     2     7  __init__@11-17@include/os/channel.py
     2     1     18     2     8  __init__@24-31@include/os/channel.py
     2     1     24     2     7  send@33-39@include/os/channel.py
     2     1     18     2     8  __init__@46-53@include/os/channel.py
     2     1     21     1     7  receive@55-61@include/os/channel.py
     5     2     33     1    12  try_receive@63-74@include/os/channel.py
     3     1     40     1    13  make_channel@77-89@include/os/channel.py
    11     1     51     1    17  __init__@15-31@include/config/args.py
    18     2    125     2    32  __init__@57-88@include/config/settings.py
    22     6    113     2    31  _parse_policy@90-120@include/config/settings.py
    60     8    323     0    79  spawn_inference_thread.inference_thread@43-121@include/thread/inference.py
    22     2     84     7   123  spawn_inference_thread@13-135@include/thread/inference.py
    76    16    378     0   134  spawn_bridge_thread.bridge_thread@26-159@include/thread/bridge.py
    14     2     67     4   162  spawn_bridge_thread@10-171@include/thread/bridge.py
   117    19    584     0   156  spawn_fusion_thread.fusion_thread@15-170@include/thread/fusion.py
    15     2     61     2   172  spawn_fusion_thread@11-182@include/thread/fusion.py
     8     3     43     1    13  __init__@67-79@include/thread/telemetry.py
     9     2     48     1    12  reset@81-92@include/thread/telemetry.py
    10     1     63     3    19  __init__@132-150@include/thread/telemetry.py
    23     4    168     4    41  record@152-192@include/thread/telemetry.py
     6     2     45     1    18  swap_buffers@194-211@include/thread/telemetry.py
     6     2     34     1    14  terminate@213-226@include/thread/telemetry.py
    28     3    139     2    33  __init__@233-265@include/thread/telemetry.py
    20     2    132     3    23  write_epoch@267-289@include/thread/telemetry.py
    46     7    221     0    61  spawn_telemetry_thread.telemetry_thread@310-370@include/thread/telemetry.py
    27     4    111     3    93  spawn_telemetry_thread@292-384@include/thread/telemetry.py
    34     2    186     3    47  __init__@20-66@include/inference/inference_engine.py
     6     2     47     1    11  run@68-78@include/inference/inference_engine.py
     9     1     58     2    17  __init__@11-27@include/queue.py
     7     2     52     2    17  try_push@29-45@include/queue.py
     4     1     46     2    11  overwrite_oldest@47-57@include/queue.py
     8     2     59     1    17  pop@59-75@include/queue.py
     2     1     22     2    10  _advance_index@77-86@include/queue.py
    28     2    109     8    52  spawn_bridge_and_inference_threads@36-87@pipeline.py
    41     6    229     0    48  main@90-137@pipeline.py
19 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     2     2.0    2.0    20.0         1  include/os/os.py
    12     6.0    3.0    42.0         1  include/os/gc.py
     3     0.0    0.0     0.0         0  include/os/__init__.py
   116    18.0    2.8   105.2         4  include/os/shm.py
    24     2.6    1.1    26.1         7  include/os/channel.py
    16    11.0    1.0    51.0         1  include/config/args.py
    21     0.0    0.0     0.0         0  include/config/__init__.py
    30     0.0    0.0     0.0         0  include/config/policy.py
    66    20.0    4.0   119.0         2  include/config/settings.py
     8     0.0    0.0     0.0         0  include/thread/__init__.py
    89    41.0    5.0   203.5         2  include/thread/inference.py
    95    45.0    9.0   222.5         2  include/thread/bridge.py
   137    66.0   10.5   322.5         2  include/thread/fusion.py
   220   18.3    3.0   100.4        10  include/thread/telemetry.py
     1     0.0    0.0     0.0         0  include/inference/__init__.py
    54    20.0    2.0   116.5         2  include/inference/inference_engine.py
     1     0.0    0.0     0.0         0  include/__init__.py
    34     6.0    1.4    47.4         5  include/queue.py
    97    34.5    4.0   169.0         2  pipeline.py
=====

```

```

=====
!!!! Warnings (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100) !!!!
=====
NLOC   CCN   token  PARAM  length  location
-----
    76    16    378     0   134 spawn_bridge_thread.bridge_thread@26-159@include/thread/bridge.py
   117    19    584     0   156 spawn_fusion_thread.fusion_thread@15-170@include/thread/fusion.py
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
   1026    18.9    3.2   104.0    41        2    0.05   0.25
=====

```

AI Usage

Google Gemini [45] was used as an assistive tool in the preparation of this report in the following ways:

- **Hardware Configuration:** to provide guidance on how to configure the Jetson Orin Nano, including initial setup, power mode configuration, thermal management, and NVIDIA utilities such as `nvpmodel` and `tegrastats`.
- **Model Generation:** to generate the Python PyTorch scripts required to export the dummy ONNX models used in the evaluation.
- **Software Engineering:** to provide reference examples for idiomatic code, and to aid in debugging issues across C++, Rust, and Python.
- **Data Analysis:** to provide guidance on how to resolve some analysis requirements in the Jupyter Notebook.
- **Explanations:** to provide simplified explanations of some advanced topics (e.g. cache-line contention and the GIL) to aid the author's understanding of the concepts.
- **Draft Review and Feedback:** to review written drafts for clarity and flow. This included adopting occasional words and phrases to improve the technical explanations.

All AI-assisted code, configurations, and explanations were reviewed, validated, and fully understood. All prose was initially drafted by the author, and may have been subsequently reviewed by the AI. While some words and phrases were adopted to refine the text, the system architecture, data, arguments, and conclusions are the author's original intellectual work.

References

- [1] X. Xue and Y. Tang, "Multimodal AI-Driven Edge Framework for Energy-Efficient Human Activity Monitoring in Smart Grids," *International Journal of Energy and Environmental Engineering*, vol. 16, no. 02, Jun. 2025, doi: 10.57647/ijeee.2025.1602.07.
- [2] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web," pp. 654–663, 1997, Accessed: May 27, 2026. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/258533.258660>
- [3] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, "The Case for VM-Based Cloudlets in Mobile Computing," *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009, doi: 10.1109/MPRV.2009.82.
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.
- [5] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019, doi: 10.1109/JPROC.2019.2918951.
- [6] *NVIDIA Jetson Linux Developer Guide, Version 36.5 GA*. Accessed: May 20, 2026. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r36.5/DeveloperGuide/index.html>
- [7] V. Peluso, R. G. Rizzo, and A. Calimera, "Performance Profiling of Embedded ConvNets under Thermal-Aware DVFS," *Electronics*, vol. 8, no. 12, 2019, doi: 10.3390/electronics8121423.
- [8] *CUDA Programming Guide, Version 13.2*. pp. 4–11. Accessed: May 09, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf>
- [9] *NVIDIA cuDNN Developer Guide, Version 8.9.0*. Accessed: May 10, 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/cudnn/archives/cudnn-890/pdf/cuDNN-Developer-Guide.pdf>
- [10] "NVIDIA TensorRT Documentation, Version 10.13.3." Accessed: May 10, 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/10.13.3/index.html>
- [11] "ONNX 1.22.0 Documentation." Accessed: May 10, 2026. [Online]. Available: <https://onnx.ai/onnx/index.html>
- [12] R. Pereira *et al.*, "Energy efficiency across programming languages: how do energy, time, and memory relate?," in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, in SLE 2017. Vancouver, BC, Canada: Association for Computing Machinery, 2017, pp. 256–267. doi: 10.1145/3136014.3136031.
- [13] MITRE Corporation, "2025 CWE Top 25 Most Dangerous Software Weaknesses," technical report, 2025. Accessed: May 31, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html
- [14] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu, "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 1, Sep. 2021, doi: 10.1145/3466642.
- [15] M. Coblenz, A. Porter, V. Das, T. Nallagorla, and M. Hicks, "A Multimodal Study of Challenges Using Rust," in *Proceedings of the Workshop on the Evaluation and Usability of Programming Languages and Tools (PLATEAU)*, Feb. 2023. Accessed: May 31, 2026. [Online]. Available: <https://mhicks.me/papers/coblenz23rust.html>
- [16] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali, "Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics," in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, in SIGCOMM '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 359–376. doi: 10.1145/3387514.3405874.
- [17] N. Rivetti, Y. Busnel, and L. Querzoni, "Load-aware shedding in stream processing systems," in *Proceedings of the 10th ACM International Conference on Distributed and Event-Based Systems*, in DEBS '16. Irvine, California: Association for Computing Machinery, 2016, pp. 61–68. doi: 10.1145/2933267.2933311.
- [18] G. Tene, "How NOT to Measure Latency." [Online]. Available: <https://www.infoq.com/presentations/latency-response-time/>
- [19] J. Evans, "A Scalable Concurrent malloc(3) Implementation for FreeBSD," 2006. Accessed: May 30, 2026. [Online]. Available: <https://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf>
- [20] "Rust Compiler RFC 1974: Global Allocators." Accessed: May 30, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/blob/master/text/1974-global-allocators.md>
- [21] "Announcing Rust 1.32.0." Accessed: May 30, 2026. [Online]. Available: <https://blog.rust-lang.org/2019/01/17/Rust-1.32.0/>
- [22] "NVIDIA Jetson Orin Nano Super Developer Kit." Accessed: May 09, 2026. [Online]. Available: <https://nvdam.widen.net/s/zkfqjmtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2>
- [23] *Arm® Cortex®-A78 Core Technical Reference Manual*. Accessed: Jul. 09, 2026. [Online]. Available: <https://developer.arm.com/documentation/101430/latest/>
- [24] "IMX219-160 Camera." Accessed: May 09, 2026. [Online]. Available: https://www.waveshare.com/wiki/IMX219-160_Camera
- [25] *BMI088, 6-axis Motion Tracking for High-performance Applications*. Accessed: May 09, 2026. [Online]. Available: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi088-ds001.pdf>
- [26] "Samsung 990 Pro NVMe SSD." Accessed: May 09, 2026. [Online]. Available: <https://www.samsung.com/uk/memory-storage/nvme-ssd/990-pro-1tb-nvme-pcie-gen-4-mz-v9p1t0bw/>
- [27] "SanDisk High Endurance microSD Card." Accessed: May 09, 2026. [Online]. Available: https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/product/memory-cards/high-endurance-uhs-i-microsd/datasheet-high-endurance-uhs-i-microsd.pdf
- [28] "JetPack SDK." Accessed: May 20, 2026. [Online]. Available: <https://developer.nvidia.com/embedded/jetpack-sdk-62>
- [29] "NVIDIA L4T JetPack 36.4.0." Accessed: Jul. 21, 2026. [Online]. Available: <https://catalog.ngc.nvidia.com/orgs/nvidia/-/containers/l4t-jetpack/r36.4.0>
- [30] J. D. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961, [Online]. Available: <https://www2.isye.gatech.edu/~spyros/courses/IE7201/Fall-13/Little-OR-paper.pdf>

- [31] G. Tene, "HdrHistogram: A High Dynamic Range Histogram." [Online]. Available: <https://github.com/HdrHistogram/HdrHistogram>
- [32] W. H. Kruskal and W. A. Wallis, "Use of Ranks in One-Criterion Variance Analysis," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 583–621, 1952, Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/2280779>
- [33] M. Tomczak and E. Tomczak, "The need to report effect size estimates revisited. An overview of some recommended measures of effect size," 2014.
- [34] O. J. Dunn, "Multiple Comparisons Using Rank Sums," *Technometrics*, vol. 6, no. 3, pp. 241–252, 1964, Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/1266041>
- [35] O. J. Dunn, "Multiple Comparisons Among Means," *Journal of the American Statistical Association*, vol. 56, no. 293, pp. 52–64, 1961, Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/2282330>
- [36] C. Spearman, "The Proof and Measurement of Association between Two Things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904, Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/1412159>
- [37] K. Pearson, "Note on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895, Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/115794>
- [38] J. Hauke and T. Kossowski, "Comparison of values of Pearson's and Spearman's correlation coefficients on the same sets of data," *Quaestiones geographicae*, vol. 30, no. 2, pp. 87–93, 2011.
- [39] B. Ray, D. Posnett, P. Devanbu, and V. Filkov, "A large-scale study of programming languages and code quality in GitHub," *Commun. ACM*, vol. 60, no. 10, pp. 91–100, Sep. 2017, doi: 10.1145/3126905.
- [40] T. Yin, "GitHub - terryyin/lizard: A simple code complexity analyser without caring about the C/C++ header files or Java imports, supports most of the popular languages. · GitHub." Accessed: May 24, 2026. [Online]. Available: <https://github.com/terryyin/lizard>
- [41] T. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976, doi: 10.1109/TSE.1976.233837.
- [42] "ctypes — A foreign function library for Python." Accessed: Jul. 09, 2026. [Online]. Available: <https://docs.python.org/3/library/ctypes.html>
- [43] "PyPI - the Python Package Index." Accessed: Jul. 25, 2026. [Online]. Available: <https://pypi.org/>
- [44] "devpi." Accessed: Jul. 25, 2026. [Online]. Available: <https://pypi.jetson-ai-lab.io/>
- [45] Google DeepMind, "Gemini." Accessed: Aug. 22, 2026. [Online]. Available: <https://gemini.google.com/>